



SKALA

HTML, CSS, JavaScript

2026.07

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. Web

2. HTML

3. CSS

4. JavaScript

1. Web

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

WEB의 정의

WEB

WEB은 인터넷 상에서 가장 인기 있는 서비스

인터넷(Internet): 전 세계의 컴퓨터를 연결한 거대한 네트워크

웹(Web): 인터넷 상에서 연결되는 페이지 기반 서비스

하이퍼텍스트(Hypertext): 문서와 문서가 링크(Link)로 촘촘하게 연결

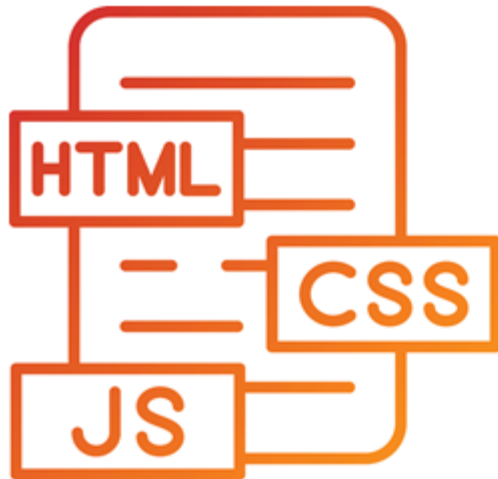


Web의 구성 요소

WEB

Web의 구성 요소 – HTML, CSS, JavaScript

기술	역할	비유
HTML	웹 페이지의 구조 와 뼈대 설계	건물의 콘크리트 벽과 기둥
CSS	웹 페이지의 시각적 스타일 및 레이아웃	건물의 외벽 페인트, 인테리어 디자인
JavaScript	웹 페이지의 동적 기능 및 상호작용 제어	건물의 엘리베이터, 자동문, 조명 스위치



Client와 Server

WEB

클라이언트(Client, 요청자)와 서버(Server, 제공자)의 주고받는 관계

클라이언트(Client, 요청자)와 서버(Server, 제공자)의 요청(Request)과 응답(Response)에 따라 Web Page(HTML, CSS, JS 등) 전달

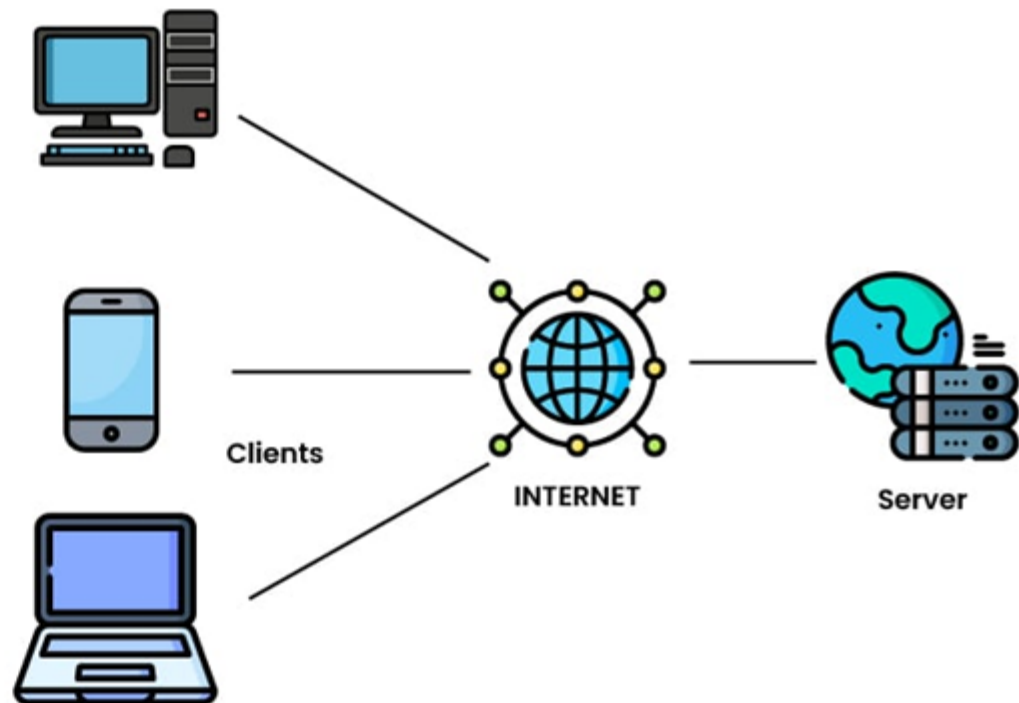
클라이언트 (Client)

- 서비스를 요청하는 주체
- 크롬(Chrome), 사파리(Safari), 엣지(Edge) 같은 웹 브라우저

서버 (Server)

- 요청을 받아 서비스를 제공(Serve)하는 주체
- 웹 페이지 데이터(HTML, 이미지 등)를 저장
- 클라이언트가 요청하면 이를 보내주는 고성능 컴퓨터

Client-Server Model in the World Wide Web



데이터가 오고 가는 4단계 과정

브라우저 주소창에 google.com을 입력하고 화면이 뜨기까지, 클라이언트와 서버가 하는 일

1. 요청 (Request) : 클라이언트에서 서버로

- 사용자가 링크를 클릭하거나 주소를 입력하면, 브라우저(클라이언트)가 HTTP라는 규칙에 맞춰 서버에 "구글 메인 화면 좀 보여줘" 하고 요청 메시지를 전달

2. 처리 (Processing) : 서버에서

- 요청을 받은 구글 서버(컴퓨터)는 해당 페이지를 보여주기 위해 필요한 데이터가 무엇인지 확인하고, 필요한 작업을 처리

3. 응답 (Response) : 서버에서 클라이언트로

- 처리가 끝나면 서버는 요청 결과(성공 또는 실패)와 함께, 브라우저가 화면을 그릴 수 있는 소스 코드(HTML, CSS, JavaScript)를 묶어서 클라이언트에게 다시 전달

4. 렌더링 (Rendering) : 클라이언트에서

- 브라우저(클라이언트)는 서버로부터 받은 HTML, CSS, JS 코드를 한 줄씩 읽어 내려가며 우리가 눈으로 보는 웹 화면으로 번역(렌더링)하여 사용자에게 보여줌

개발 영역의 분리

WEB

프론트엔드 vs 백엔드

프론트엔드 (Front-End) / 클라이언트 사이드

- 사용자의 눈에 직접 보이는 화면(브라우저 영역)을 개발
- 핵심 기술: HTML, CSS, JavaScript, React, Vue 등

백엔드 (Back-End) / 서버 사이드

- 눈에 보이지 않는 뒷선(서버 영역)에서 데이터를 저장, 가공, 인증하는 비즈니스 로직을 개발
- 핵심 기술: Java/Spring, Python/Django, Node.js, MySQL 등

URL(Uniform Resource Locator)은 특정 리소스의 위치와 접근 방법을 나타냄

URL의 구조

`https://www.example.com:443/products/shoes?size=270&color=black#reviews`

Query String(쿼리 스트링)의 활용

- 클라이언트 상에서 서버로 데이터를 전달할 때

예) 쇼핑몰에서 상품을 검색하거나 필터링할 때 (?search=macbook&page=2)

구성 요소	이름	역할	예시에서의 값
https://	Protocol (프로토콜)	어떤 규칙으로 데이터를 주고받을 것인가? (웹은 주로 http나 https 사용)	https
www.example.com	Domain Name (도메인)	요청을 보낼 서버의 컴퓨터 이름(주소). (원래는 192.168.0.1 같은 숫자로 된 IP 주소지만, 사람이 읽기 쉽게 문자로 바꾼 것)	[www.example.com] (https://www.example.com)
:443	Port (포트 번호)	서버 컴퓨터 내의 어떤 문(통로)으로 들어갈 것인가? (https의 기본 포트는 443이라 보통 생략됨)	443
/products/shoes	Path (경로)	서버 컴퓨터 안에서 내가 원하는 파일이나 자원의 위치 (마치 컴퓨터의 폴더 경로와 같음)	/products/shoes
?size=270...	Query String (쿼리 스트링)	서버에 전달하는 추가 조건(파라미터). ? 뒤에 Key=Value 형태로 붙으며, 여러 개일 땐 & 로 연결	size=270&color=black
#reviews	Fragment (앵커)	문서 내의 **특정 위치(북마크)**로 바로 스크롤 이동하게 만드는 꼬리표	#reviews

HTTP

HTTP(HyperText Transfer Protocol)는 뜻 그대로 '하이퍼텍스트(웹 문서)를 전송하기 위한 통신 규약(약속)'

요청과 응답(Request / Response)

- 클라이언트의 요청에 서버가 응답하는 방식으로 동작

무상태성(Stateless)

- 서버는 클라이언트의 과거 상태를 기억하지 못함. 매번 새로운 요청과 응답으로 처리

HTTP 요청 메서드 (HTTP Methods)

클라이언트가 서버에 요청을 보낼 때, "내가 지금 무엇을 하려고 하는지"를 간단히 설명

메서드	역할	실무 비유 (게시판 기준)
GET	서버로부터 데이터를 가져올 때 사용 (조회)	게시글 읽기, 검색하기
POST	서버에 새로운 데이터를 제출할 때 사용 (생성)	새 글 쓰기, 회원가입하기
PUT / PATCH	서버의 기존 데이터를 수정할 때 사용 (수정)	작성한 글 수정하기
DELETE	서버의 데이터를 삭제할 때 사용 (삭제)	글 삭제하기

HTTP 응답 상태 코드 (Status Code)

서버가 클라이언트에 응답을 보낼 때, 처리 결과를 상태 코드로 보냄

코드 대역	의미	대표적인 예시
2xx (성공)	요청이 정상적으로 처리됨	200 OK: 성공!
3xx (리다이렉션)	요청 완료를 위해 다른 주소로 이동해야 함	301 Moved Permanently: 페이지 주소 바뀜
4xx (클라이언트 오류)	요청 보낸 사람의 문법이나 주소가 잘못됨	404 Not Found: 그런 주소나 페이지는 없음 401 Unauthorized: 로그인 안 함
5xx (서버 오류)	요청은 잘 왔는데 받는 사람 컴퓨터가 잘못됨	500 Internal Server Error: 서버 코드 에러

컴퓨터가 서로를 식별하고 찾아가기 위해 사용하는 실제 주소

IP 주소란?

- 인터넷에 연결된 모든 장치(컴퓨터, 스마트폰, 서버, 심지어 IoT 가전제품까지)가 가진 고유한 식별 번호

IPv4와 IPv6 (주소 체계의 진화)

- IPv4 (Internet Protocol version 4)
 - 형태: 211.234.23.10 처럼 0~255 사이의 숫자 4조각이 점(.)으로 구분된 형태
 - 한계: 이론상 약 43억 개의 주소 가능. 주소를 갖게 되는 디바이스의 폭증으로 주소가 고갈되는 문제가 발생
- IPv6 (Internet Protocol version 6)
 - 형태: 2001:0db8:85a3:0000:0000:8a2e:0370:7334 처럼 16진수 숫자가 콜론(:)으로 구분된 복잡한 형태
 - 한계: IPv4의 주소 고갈 문제를 해결하기 위해 등장, 거의 무한대에 가까운 주소를 제공

공인 IP와 사설 IP (Public vs Private)

- 공인 IP (Public IP): 인터넷 세상에서 유일무이하게 부여된 진짜 주소, 전 세계에서 이 주소로 내 컴퓨터에 직접 접속 가능
- 사설 IP (Private IP): 공유기(라우터)가 집이나 회사 내부에서 자기들끼리 구별하기 위해 가짜로 나눠준 내부용 주소, 주로 192.168.x.x 나 10.x.x.x로 시작, 외부 인터넷 세상에서는 이 주소로 나에게 직접 찾아올 수 없음

컴퓨터가 서로를 식별하고 찾아가기 위해 사용하는 실제 주소

DNS(Domain Name System)란?

- DNS(도메인 네임 시스템)는 사람이 읽을 수 있는 문자 주소(Domain Name)를 컴퓨터가 이해할 수 있는 숫자 주소(IP Address)로 번역해주는 거대한 시스템

1. Local DNS 서버 확인

- 컴퓨터는 먼저 본인이 가입한 ISP의 Local DNS 서버에게 `www.example.com`의 IP 주소를 확인
- 만약 기존에 저장(캐싱)된 정보가 없다면 다음 단계로 넘어감

2. Root 네임 서버 (.)

- Local DNS 서버는 Root 서버에게 질의
- Root 서버는 끝자리(.com)를 보고 .com 주소를 관리하는 TLD 서버 주소를 알려줌

3. TLD(Top-Level Domain) 네임 서버 (.com)

- Local DNS 서버는 안내받은 .com TLD 서버로 질의
- TLD 서버는 `example.com`은 특정 호스팅 업체(예: 가비아, AWS 등)가 관리하니 그 네임 서버의 주소를 알려줌

4. Authoritative(권한 있는) 네임 서버

- 마지막으로 도메인을 실제 구매하고 등록한 권한 있는 네임 서버에 도달
- 이 서버는 마침내 `www.example.com`의 진짜 IP 주소는 `XXX.XXX.XXX.XXX`라고 최종 주소를 알려줌

5. IP 주소 전달 및 캐싱

- 정답을 받아온 Local DNS 서버는 이 주소를 내 컴퓨터(브라우저)에 전달하고, 나중을 위해 자신의 메모리에 일정 시간 저장(Caching)
- 브라우저는 드디어 알아낸 진짜 IP 주소로 진짜 HTTP 요청을 보냄



- 1 DTD : 문서형 정의 (<!DOCTYPE html>는 HTML5 문서라는 것을 의미한다.)
- 2 html : HTML 문서의 시작과 끝
- 3 head : 문서 정보 정의
- 4 body : 웹 브라우저에 표시할 내용

카테고리	태그	설명
문서 구조	<html>	HTML 문서의 루트 요소로, 문서의 시작과 끝을 표시
	<head>	메타데이터(문서 제목, 스타일, 스크립트 등)를 정의
	<title>	브라우저 탭에 표시되는 문서의 제목
	<body>	문서의 본문 콘텐츠를 정의
Metadata	<meta>	문서의 메타데이터를 정의하며, SEO 및 브라우저 동작에 영향을 줌
	<link>	외부 리소스(CSS 파일 등)와의 관계를 정의
	<script>	JavaScript 코드를 삽입하거나 외부 스크립트를 연결

카테고리	태그	설명
Text/Contents	<h1> ~ <h6>	제목(Heading)을 정의하며, <h1>이 가장 크고 <h6>이 가장 작음
	<p>	단락을 정의
	 	줄바꿈을 삽입
		인라인 요소, 텍스트를 묶어 스타일링이나 스크립트를 적용할 때 사용
		중요성을 나타내는 텍스트를 굵게 표시
		강조된 텍스트를 기울임꼴로 표시
	<a>	하이퍼링크를 생성하며, href 속성을 통해 링크 대상을 지정
	<hr>	수평선
	<mark>	형광펜 강조
	<small>	작은 글씨
	<blockquote>	인용 문단
	<cite>	출처 인용
	<code>	코드 조각 표현
	<pre>	공백/줄바꿈 유지한 채 코드나 텍스트 표시
	<div>	의미 없는 구획(block), 스타일/스크립트 용도

카테고리	태그	설명
List Tag		순서 없는 목록(Unordered List)을 정의
		순서 있는 목록(Ordered List)을 정의
		목록의 항목을 정의: 및 에서 모두 사용
	<dl>	설명 목록
	<dt>	설명 제목
	<dd>	설명 내용
Table Tag	<table>	테이블(표 양식)을 정의
	<thead>	표의 머리글 부분
	<tbody>	표의 본문
	<tfoot>	표의 바닥글
	<tr>	테이블 행(Row)을 정의
	<th>	테이블의 헤더 셀을 정의하며, 기본적으로 텍스트를 굵게 표시하고 가운데 정렬
	<td>	테이블 데이터 셀을 정의

카테고리	태그	설명
Form Tag	<form>	사용자가 데이터를 입력하여 서버로 전송할 수 있는 폼을 정의
	<input>	다양한 입력 필드를 정의(텍스트, 비밀번호, 체크박스 등)
	<button>	버튼
	<label>	입력 필드에 대한 설명을 제공
	<textarea>	여러 줄의 텍스트 입력 필드
	<select>	드롭다운 선택 필드
	<option>	선택 항목
	<fieldset>	입력 항목 그룹화
	<legend>	그룹 설명 제목

카테고리	태그	설명
Media Tag		이미지를 삽입하며, src 속성으로 이미지 경로를, alt 속성으로 대체 텍스트를 지정
	<audio>	오디오 콘텐츠를 삽입
	<video>	비디오 콘텐츠를 삽입
	<source>	<audio>나 <video>에 미디어 소스를 지정
	<track>	자막 트랙
	<figure>	미디어와 캡션 묶음
	<figcaption>	설명 캡션

카테고리	태그	설명
Semantic Tag	<header>	문서나 섹션의 머리글을 정의
	<footer>	문서나 섹션의 바닥글을 정의
	<main>	문서의 주 콘텐츠
	<article>	독립적인 콘텐츠를 정의
	<section>	문서의 주제를 정의하는 섹션
	<aside>	부가적인 콘텐츠(예: 사이드바)를 정의.
	<nav>	네비게이션 링크를 포함하는 섹션을 정의

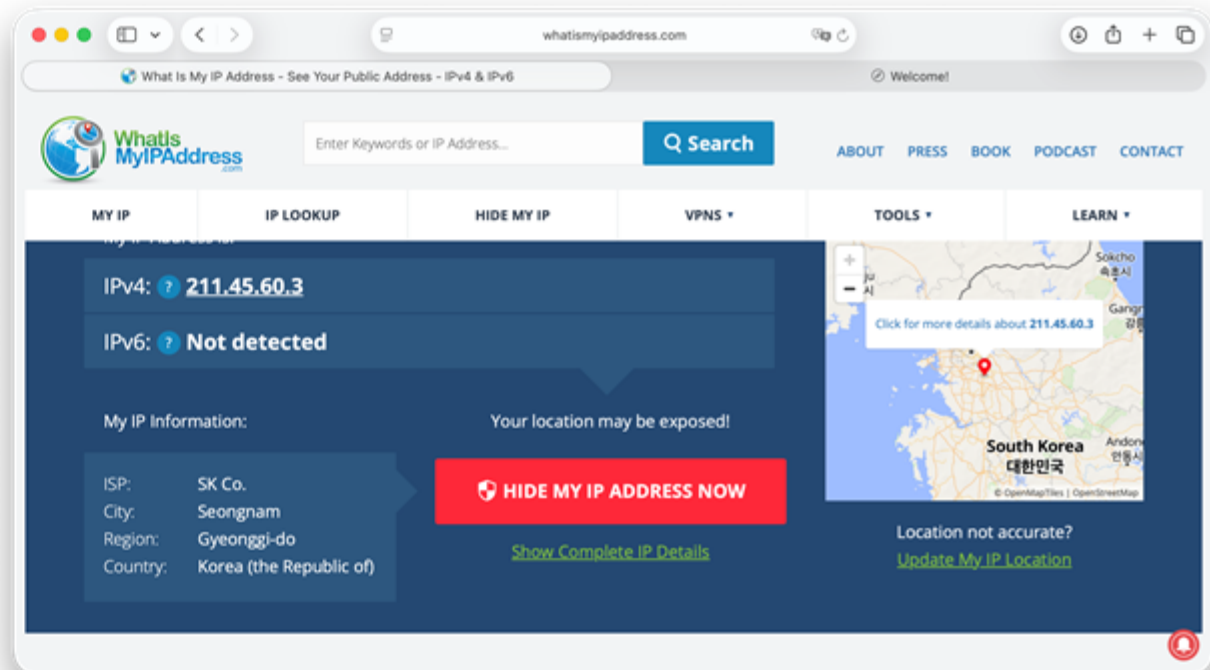
[실습] IP 확인

WEB

사용 중인 컴퓨터의 IP를 확인해보기

1. 터미널에서 ifconfig 명령어를 통하여 IP 확인해보기
2. <https://whatismyipaddress.com/> 를 통하여 IP 확인해보기
3. 두 결과를 비교해보기

```
status: inactive
ap1: flags=8822<BROADCAST,SMART,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether b2:52:9d:c0:10:8a
    media: autoselect (none)
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=6460<TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
    ether 96:0d:c1:be:a2:85
    inet6 fe80::c5b:f20:794:8092%en0 prefixlen 64 secured scopeid 0xb
    inet 10.250.202.17 netmask 0xfffffc00 broadcast 10.250.203.255
    nd6 options=201<PERFORMNUD,DAD>
    media: autoselect
    status: active
awd10: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
```

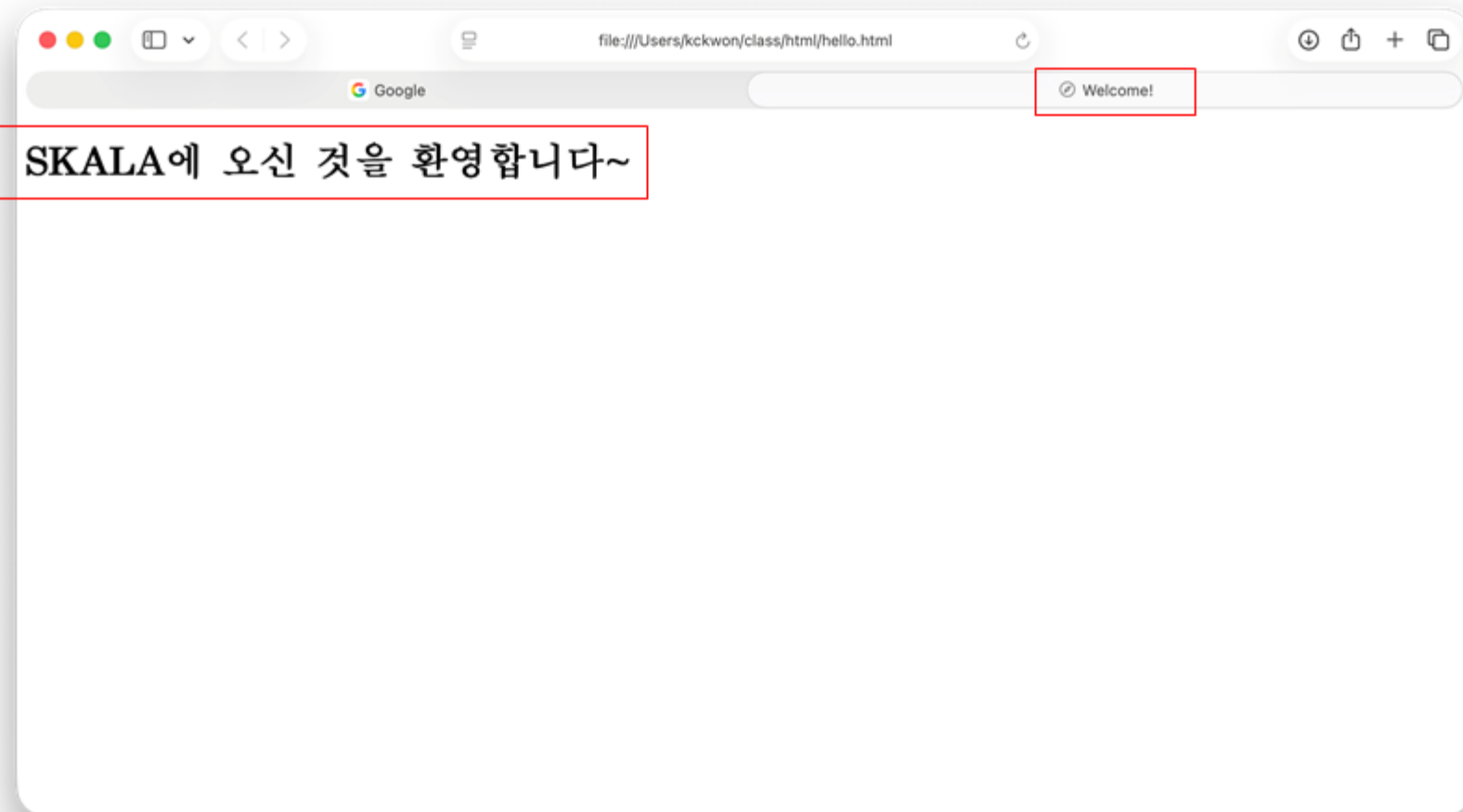


[실습] HTML 페이지 만들기

WEB

HTML 페이지 만들어 보기

HTML Page Structure 슬라이드를 참고하여 HTML 페이지 만들어 보기



2. HTML

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

- An HTML element is defined by a start tag, some content, and an end tag
 - `<tagname>` Content goes here... `</tagname>`
- The HTML element is everything from the start tag to the end tag

Start Tag	Element Content	End Tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>	<i>none</i>	<i>none</i>

- HTML elements with no content are called empty elements.
- HTML elements can be nested (this means that elements can contain other elements).

```
<html>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
  </body>
</html>
```

- HTML is Not Case Sensitive

- HTML attributes provide additional information about HTML elements.

- All HTML elements can have attributes
- Attributes provide additional information about elements
- Attributes are always specified in the start tag
- Attributes usually come in name/value pairs like: name="value"

```
<a href="https://www.w3schools.com">Visit W3Schools</a>  
  
<p style="color:red;">This is a red paragraph.</p>
```

- Double quotes around attribute values are the most common in HTML, but single quotes can also be used.

```
<p title='John "ShotGun" Nelson'>  
<p title="John 'ShotGun' Nelson">
```

- 다음 속성은 모든 HTML Element에서 사용되는 속성이다.

Attribute	설명	예시
id	요소의 고유 식별자 - JS, CSS 등에서 참조할 때 사용	<code><div id="title">... </div></code>
class	CSS나 JS에서 참조 가능한 클래스 이름 (여러 개 가능)	<code><p class="red-color">... </p></code>
style	인라인 스타일을 지정	<code><h1 style="color:red;"></h1></code>
title	엘리먼트의 추가정보 (마우스를 위에 올리면 툴팁으로 표시)	<code></code>
lang	엘리먼트에 사용한 텍스트의 언어정보를 지정	<code><html lang="ko"></code>
hidden	요소를 화면에 표시하지 않음 (렌더링 X)	<code><div hidden>비밀 정보</div></code>
tabindex	키보드 탭 순서 지정 (작은 숫자가 먼저)	<code><button tabindex="1">확인</button></code>
draggable	요소를 드래그할 수 있는지 여부 설정 (true / false)	<code></code>
spellcheck	입력된 텍스트의 맞춤법 검사 여부 (true / false)	<code><textarea spellcheck="true" lang="kr"></textarea></code>
contenteditable	사용자가 콘텐츠를 편집할 수 있도록 설정 (true / false)	<code><div contenteditable="true">편집 가능</div></code>
dir	텍스트 방향 설정 (ltr, rtl, auto)	<code><p dir="rtl">שלום</p></code>
lang	콘텐츠 언어 지정 (en, ko, fr 등)	<code><p lang="en">Hello</p></code>
data-*	사용자가 임의의 속성을 생성	<code><p data-name="spiderMan" data-hero="true">... </p></code>

- The **id** attribute specifies a **unique id** for an HTML element.
- The value of the id attribute **must be unique** within the HTML document.

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>

<h2>The id Attribute</h2>
<p>Use CSS to style an element with the id "myHeader":</p>

<h1 id="myHeader">My Header</h1>

</body>
</html>
```

The id Attribute

Use CSS to style an element with the id "myHeader":

My Header

HTML Attributes - class

- The **class** attribute is often used to point to a class name in a style sheet.
- It can also be used by JavaScript to access and manipulate elements with the specific class name.

```
<!DOCTYPE html>
<html>
<head>
<style>
.note {
  font-size: 120%;
  color: red;
}
</style>
</head>
<body>

<h1>My <span class="note">Important</span> Heading</h1>
<p>This is some <span class="note">important</span> text.</p>

</body>
</html>
```

My **Important** Heading

This is some **important** text.

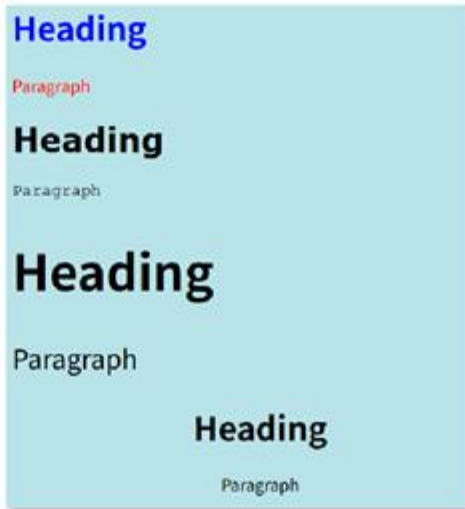
- The HTML **style** attribute is used to add styles to an element, such as color, font, size, and more.
 - `<tagname style="property:value;">`
 - The property is a CSS property. The value is a CSS value.
- The CSS **background-color** property defines the background color for an HTML element.

```
<body style="background-color:powderblue;">  
  <h1 style="background-color:lightgray;">제목</h1>  
  <p style="background-color:tomato;">문단</p>  
</body>
```



- CSS의 `color`, `font-family`, `font-size`, `text-align` 같은 property를 통해 HTML을 꾸밀 수 있다.

```
<h1 style="color:blue;">Heading</h1>  
<p style="color:red;">Paragraph</p>  
<h1 style="font-family:verdana;">Heading</h1>  
<p style="font-family:courier;">Paragraph</p>  
<h1 style="font-size:300%;">Heading</h1>  
<p style="font-size:160%;">Paragraph</p>  
<h1 style="text-align:center;">Heading</h1>  
<p style="text-align:center;">Paragraph</p>
```



HTML Headings - <h1>~<h6>

HTML

- **HTML headings are titles or subtitles** that you want to display on a webpage.
- 검색엔진(Search engines)이 웹페이지의 구조와 내용을 파악하는데도 사용한다.

```
<!DOCTYPE html>
<html>
<body>

<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>

</body>
</html>
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

HTML Paragraphs - <p>

- The HTML `<p>` element defines a **paragraph**.
- A paragraph always starts on a new line.

```
<p>This is a paragraph.</p>
```

```
<p>This is another paragraph.</p>
```

This is a paragraph.

This is another paragraph.

- The browser will automatically remove any extra spaces and lines when the page is displayed

```
<p>
```

```
  이 문단은
```

```
  소스 코드에서 많은 줄 바꿈이 있지만
```

```
  브라우저는 이를 무시합니다.
```

```
</p>
```

```
<p>
```

```
  이 문단은
```

```
  브라우저는 이를
```

```
  소스 코드에서 많은 공백이 있지만
```

```
  무시합니다.
```

```
</p>
```

이 문단은 소스 코드에서 많은 줄 바꿈이 있지만 브라우저는 이를 무시합니다.

이 문단은 소스 코드에서 많은 공백이 있지만 브라우저는 이를 무시합니다.

HTML Paragraphs - <hr>/

- The **<hr>** tag defines a **thematic break** in an HTML page
 - 대부분 수평선을 그려서 시각적으로 구분된다.
- The HTML **
** element defines a **line break**.
- <hr>과
 태그는 empty tag로 end tag가 필요하지 않다.

```
<!DOCTYPE html>
<html>
<body>
  <h1>This is heading 1</h1>
  <p>This is some text.</p>
  <hr>
  <h2>This is heading 2</h2>
  <p>This is some other text.</p>
  <hr>
  <p>This is<br>a paragraph<br>with line breaks.</p>
</body>
</html>
```

This is heading 1

This is some text.

This is heading 2

This is some other text.

This is
a paragraph
with line breaks.

- 모든 HTML Element는 기본 표시(display) 값을 가지고 있다.

구분	Block Elements	Inline Elements
디스플레이	화면에 새 줄에서 시작하며, 가로 폭을 가능한 한 최대로 차지	텍스트 또는 다른 인라인 요소와 같은 줄에서 표시
영역 크기	기본적으로 전체 폭을 차지	콘텐츠의 크기만큼 공간을 차지
배치	위/아래로 쌓이며(수직 정렬) 다른 요소와 겹치지 않음	다른 인라인 요소와 가로로 나란히 배치
주요 태그	<div>, <p>, <h1> ~ <h6>, , , , <table>, <form>, <section>, <article>, <header>, <footer>, <nav> 등	, <a>, , , , <i>, , <input>, <label>, <button>, <small>, <code>, <mark> 등
CSS 적용	width, height, margin, padding 모두 적용 가능	width와 height는 무시, margin과 padding은 수평 방향만 적용
용도	레이아웃 구성 및 큰 영역을 나타낼 때 사용	텍스트나 소규모 콘텐츠 강조, 링크, 스타일 적용 시 사용

- element는 텍스트 또는 문서의 일부를 표시하는 데 사용되는 inline container이다.
- 요소에는 필수 속성이 없지만 style, class, id 속성이 일반적으로 사용되며, CSS와 함께 사용하면서 텍스트의 특정 부분에 스타일을 지정할 수 있다.

```
<!DOCTYPE html>
<html>
<body>
  <h1>The span element</h1>
  <p>My mother has <span style="color:blue;font-weight:bold;">blue</span> eyes and my father has <span
style="color:darkolivegreen;font-weight:bold;">dark green</span> eyes.</p>
</body>
</html>
```

The span element

My mother has blue eyes and my father has dark green eyes.

- The **<div>** element is used as a **container for other HTML elements**.
- The **<div>** element is by default a **block element**, meaning that it takes all available width, and comes with line breaks before and after.

```
<!DOCTYPE html>
<html>
<body>
<h1>Multiple DIV Elements</h1>
<div style="background-color:#FFF4A3;">
  <h2>London</h2>
  <p>London is the capital city of England.</p>
  <p>London has over 9 million inhabitants.</p>
</div>
<div style="background-color:#FFC0C7;">
  <h2>Oslo</h2>
  <p>Oslo is the capital city of Norway.</p>
  <p>Oslo has over 700,000 inhabitants.</p>
</div>
<div style="background-color:#D9EEE1;">
  <h2>Rome</h2>
  <p>Rome is the capital city of Italy.</p>
  <p>Rome has over 4 million inhabitants.</p>
</div>
<p>CSS styles are added to make it easier to separate the divs,
and to make them more pretty:</p>
</body>
</html>
```

Multiple DIV Elements

London

London is the capital city of England.

London has over 9 million inhabitants.

Oslo

Oslo is the capital city of Norway.

Oslo has over 700,000 inhabitants.

Rome

Rome is the capital city of Italy.

Rome has over 4 million inhabitants.

CSS styles are added to make it easier to separate the divs, and to make them more pretty:)

- The HTML <a> tag defines a **hyperlink**.
- href 속성은 링크의 목적지 URL을 지정한다.

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

[Visit W3Schools.com!](https://www.w3schools.com/)

- target 속성은 링크된 문서가 열리는 공간을 지정한다.
 - _self - Default. Opens the document in the same window/tab as it was clicked
 - _blank - Opens the document in a new window or tab
 - _parent - Opens the document in the parent frame
 - _top - Opens the document in the full body of the window

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

- Link Bookmarks
 - Bookmarks can be useful if a web page is very long.

```
<a href="#C4">Jump to Chapter 4</a>
```

```
...
```

```
<h2 id="C4">Chapter 4</h2>
```

```
...
```

HTML Text Formatting - ```<strong>``<i>``<em>``<small>``<mark>`...

- Formatting elements were designed to display special types of text

```
<body>
  <p><b>This text is bold</b></p>
  <p><strong>This text is important!</strong></p>
  <p><i>This text is italic</i></p>
  <p><em>This text is emphasized</em></p>
  <p><small>This is some smaller text.</small></p>
  <p>Do not forget to buy <mark>milk</mark> today.</p>
  <p>My favorite color is <del>blue</del> red.</p>
  <p>My favorite color is <del>blue</del> <ins>red</ins>.</p>
  <p>This is <sub>subscripted</sub> text.</p>
  <p>This is <sup>superscripted</sup> text.</p>
</body>
```

This text is bold

This text is important!

This text is italic

This text is emphasized

This is some smaller text.

Do not forget to buy **milk** today.

My favorite color is ~~blue~~ red.

My favorite color is ~~blue~~ red.

This is _{subscripted} text.

This is ^{superscripted} text.

HTML Quotations - <blockquote><abbr>

- The HTML <blockquote> element defines a section that is quoted from another source.

```
<body>
  <p>Here is a quote from WWF's website:</p>
  <blockquote cite="http://www.worldwildlife.org/who/index.html">
    For 60 years, WWF has worked to help people and nature thrive. As the
    world's leading conservation organization, WWF works in nearly 100 countries. At
    every level, we collaborate with people around the world to develop and deliver
    innovative solutions that protect communities, wildlife, and the places in which
    they live.
  </blockquote>
</body>
```

Here is a quote from WWF's website:

For 60 years, WWF has worked to help people and nature thrive. As the world's leading conservation organization, WWF works in nearly 100 countries. At every level, we collaborate with people around the world to develop and deliver innovative solutions that protect communities, wildlife, and the places in which they live.

- The HTML <abbr> tag defines an abbreviation or an acronym, like "HTML", "CSS", "Mr.", "Dr.", "ASAP", "ATM".

```
</head>
<body>
  <p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
</body>
```

The WHO was founded in 1948.

World Health Organization

HTML Lists - Unordered Lists

- An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.
- The CSS `list-style-type` property is used to define the style of the list item marker.

```
<ul style="list-style-type:disc;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:circle;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:square;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:none;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

- Coffee
- Tea
- Milk

- Coffee
- Tea
- Milk

- Coffee
- Tea
- Milk

Coffee
Tea
Milk

HTML Lists - Ordered Lists

- An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.
- The `type` attribute of the `<ol>` tag, defines the type of the list item marker:

```
<ol type="1">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>  
<ol type="A">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>  
<ol type="a">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>  
<ol type="I">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>  
<ol type="i">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

1. Coffee
2. Tea
3. Milk

A. Coffee
B. Tea
C. Milk

a. Coffee
b. Tea
c. Milk

I. Coffee
II. Tea
III. Milk

i. Coffee
ii. Tea
iii. Milk

HTML Lists - Nested HTML Lists

- Lists can be nested (list inside list)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
</head>
<body>
<ol>
  <li>Coffee</li>
  <li>Tea
    <ul>
      <li>Black tea</li>
      <li>Green tea</li>
    </ul>
  </li>
  <li>Milk</li>
</ol>
</body>
</html>
```

1. Coffee
2. Tea
 - Black tea
 - Green tea
3. Milk

HTML Lists - Description Lists <dl><dt><dd>

HTML

- <dl>: 정의 리스트 전체를 감싸는 태그로 정의할 항목들의 목록을 포함
- <dt>: 목록 항목의 제목으로 주로 용어나 항목 이름을 입력
- <dd>: dt에서 정의된 항목에 대한 상세 설명

```
<body>
<dl>
  <dt>HTML</dt>
  <dd>Hypertext Markup Language, 웹 페이지를 만드는 데 사용되는 마크업 언어입니다.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets, 웹 페이지의 스타일을 정의하는 언어입니다.</dd>
  <dt>JavaScript</dt>
  <dd>웹 페이지의 동적인 기능을 구현하는 프로그래밍 언어입니다.</dd>
</dl>
</body>
```

HTML

Hypertext Markup Language, 웹 페이지를 만드는 데 사용되는 마크업 언어입니다.

CSS

Cascading Style Sheets, 웹 페이지의 스타일을 정의하는 언어입니다.

JavaScript

웹 페이지의 동적인 기능을 구현하는 프로그래밍 언어입니다.

- A table in HTML consists of table cells inside rows and columns.

<table>	표 전체를 감싸는 태그
<thead>	표의 머리글(제목 행)을 정의합니다. <tr>과 <th>로 구성
<tbody>	실제 데이터 행을 담는 부분입니다. <tr>과 <td>로 구성됩니다.
<tfoot>	표의 바닥글을 정의합니다. 합계나 요약 정보를 넣는 데 사용됩니다.
<tr>	표의 한 행(Row)을 의미합니다.
<th>	머리글 셀(Header cell)을 정의하며, 기본적으로 굵은 글씨와 가운데 정렬입니다.
<td>	일반 셀(Data cell)을 정의합니다.

```
<h2>TR elements define table rows</h2>
<table style="width:100%">
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>
```

TR elements define table rows

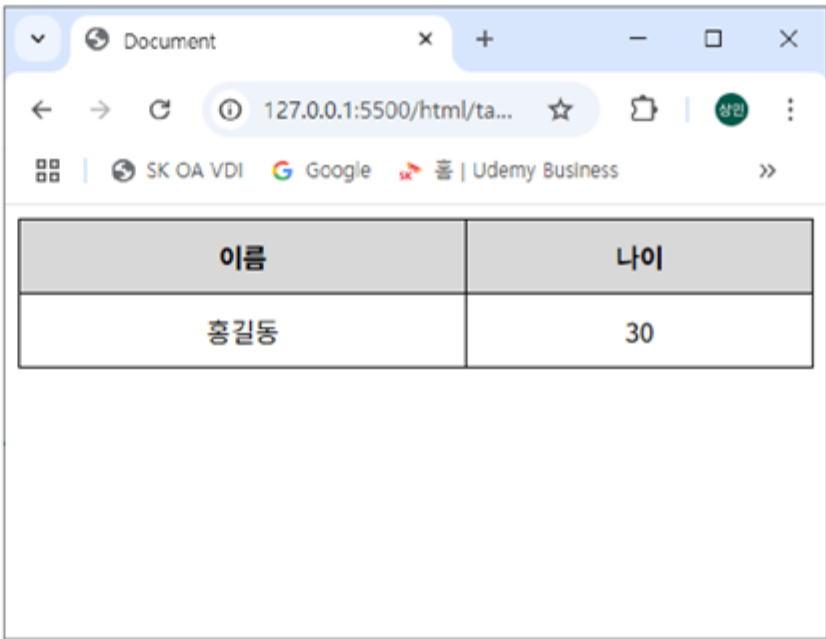
Emil	Tobias	Linus
16	14	10

HTML Tables - <thead><tbody><tfoot>

HTML

- 단순 예제나 소규모 표에서는 <tr><th>만으로 사용 가능.
- 접근성, 유지보수, 스타일링, 상호작용 구현 측면에서는 반드시 <thead>, <tbody>, <tfoot>을 구분해서 사용

```
<body>
  <table>
    <thead>
      <tr>
        <th>이름</th>
        <th>나이</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>홍길동</td>
        <td>30</td>
      </tr>
    </tbody>
  </table>
</body>
```



A screenshot of a web browser window showing the rendered HTML table. The browser's address bar displays the URL 127.0.0.1:5500/html/ta... and the page title is 'Document'. The table has two columns: '이름' (Name) and '나이' (Age). The first row contains the values '홍길동' (Hong Gildong) and '30'.

이름	나이
홍길동	30

- Use CSS to make your tables look better.

주요 항목	HTML 속성 사용 (지양)	CSS 사용 (권장)
테두리	border	border, border-collapse
셀 간격	cellspacing	border-spacing
셀 안쪽 여백	cellpadding	padding
정렬	align	margin, text-align
배경색	bgcolor	background-color

```
<head>
<style>
table {border-collapse: collapse; width: 100%;}
th,td {text-align: left; padding: 8px;}
tr:nth-child(even) {background-color: #D6EEEE;}
</style>
</head>
<body>
<h2>Zebra Striped Table</h2>
<table>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Points</th>
  </tr>
  ...
```

Zebra Striped Table

First Name	Last Name	Points
Peter	Griffin	\$100
Lois	Griffin	\$150
Joe	Swanson	\$300
Cleveland	Brown	\$250

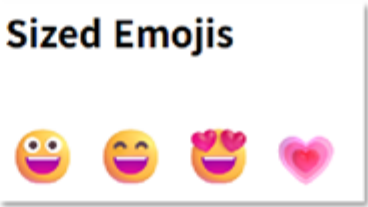
- Emojis are **letters (characters)** from the UTF-8 (Unicode) character set
 - 이모지(Emoji): 감정, 사물, 개념 등을 나타내는 그림문자
 - 유니코드(Unicode, 국제 문자 인코딩 표준의 일부)로 관리
 - 일반 텍스트처럼 전송되며 유니코드 코드포인트(U+1F600형식)로 표현
- Emoji 역사



1999년	일본 통신사 NTT 도코모가 세계 최초로 휴대폰용 이모지 도입 (176개)
2010년	유니코드 6.0에 처음으로 722개의 이모지가 공식 포함
2016년	성별(남성/여성/성중립), 직업 관련 이모지 추가
2025년	3,800개 이상의 이모지가 등록되어 있으며, 매년 유니코드에서 새로운 이모지 승인

- Emoji는 문자임으로 그냥 복사하고 붙여넣기하면 다른 문자들처럼 HTML에서 사용될 수 있다.

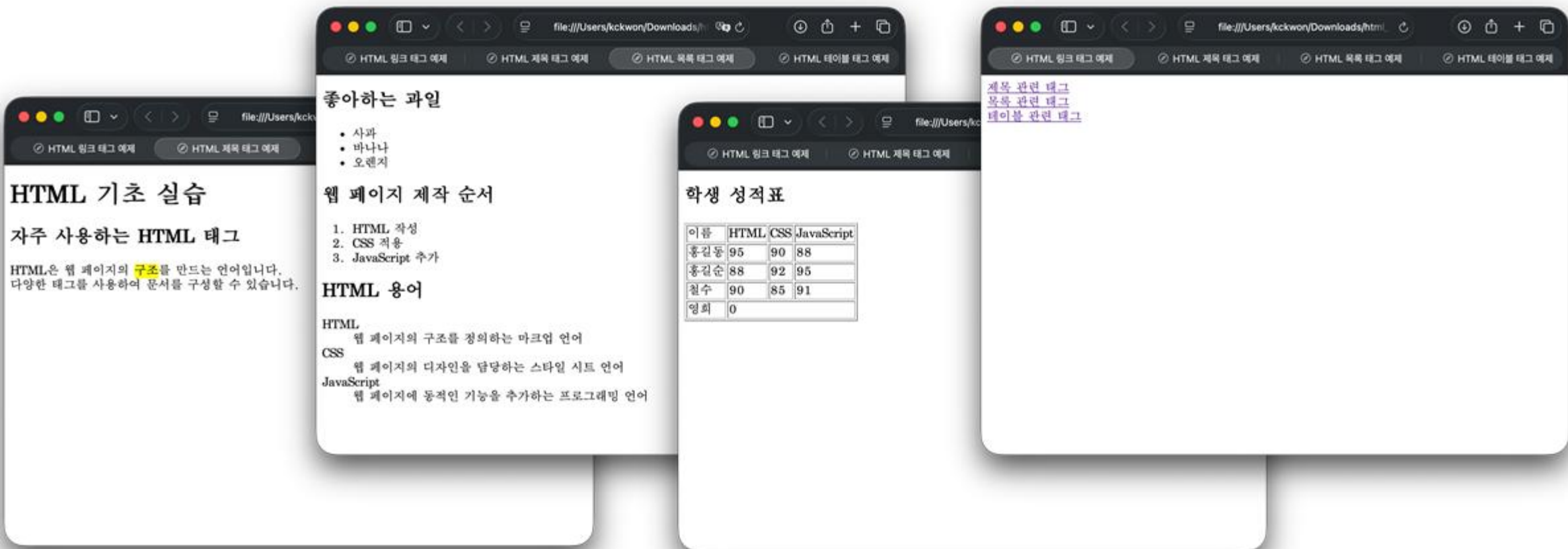
```
<!DOCTYPE html>
<html>
<meta charset="UTF-8">
<body>
  <h1>Sized Emojis</h1>
  <p style="font-size:48px">&#128512; &#128516; &#128525; &#128151;</p>
</body>
</html>
```



[실습] HTML 실습

HTML

HTML을 이용한 실습



- An HTML form is used to collect user input.
- The user input is most often sent to a server for processing.
- The HTML `<form>` element is used to create an HTML form for user input

```
<form>
.
form elements
.
</form>
```

- The `<form>` element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

```
<h2>HTML Forms</h2>
<form action="signUpResult.php" method="get">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

HTML Forms

First name:

Last name:

Attribute	Description
accept-charset	Specifies the character encodings used for form submission
action	Specifies where to send the form-data when a form is submitted
autocomplete	Specifies whether a form should have autocomplete on or off
enctype	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
method	Specifies the HTTP method to use when sending form-data
name	Specifies the name of the form
novalidate	Specifies that the form should not be validated when submitted
rel	Specifies the relationship between a linked resource and the current document
target	Specifies where to display the response that is received after submitting the form

- The HTML <form> element can contain one or more of the following form elements

태그	설명	주요 속성
<input>	사용자로부터 다양한 형태의 입력을 받음	type, name, value, placeholder, required 등
<label>	입력 필드에 대한 텍스트 설명을 제공	for (ID와 연결)
<textarea>	여러 줄의 텍스트 입력	rows, cols, placeholder, name
<select>	드롭다운 목록을 제공	name, multiple, required
<option>	<select> 안에서 선택 가능한 항목	value, selected
<button>	클릭 가능한 버튼 생성	type (submit, reset, button)
<fieldset>	관련 입력 필드를 그룹화	없음
<legend>	<fieldset>에 대한 제목 제공	없음
<datalist>	<input>에 자동완성 옵션 제공	<option>과 함께 사용
<output>	계산 결과 등의 출력용 필드	for, name

- Here are the different input **types** you can use in HTML

태그	설명	주요 속성
text	일반 텍스트 입력 필드	<input type="text" placeholder="이름 입력">
password	입력 내용을 숨기는 비밀번호 필드	<input type="password" placeholder="비밀번호 입력">
email	이메일 주소 입력 필드. 이메일 형식 검증 제공	<input type="email" placeholder="이메일 입력">
number	숫자 입력 필드. min, max, step 속성을 사용 가능	<input type="number" min="1" max="100" step="1">
tel	전화번호 입력 필드. 전화번호 형식 검증은 없음	<input type="tel" placeholder="전화번호 입력">
url	URL 입력 필드. URL 형식 검증 제공	<input type="url" placeholder="https://example.com">
color	색상 선택기 제공	<input type="color">
date	날짜 선택 필드 (YYYY-MM-DD 형식)	<input type="date">
checkbox	체크박스. 하나 이상의 선택지를 선택할 수 있음	<input type="checkbox">
radio	라디오 버튼. 하나의 그룹 내에서 하나만 선택 가능	<input type="radio" name="group">
file	파일 선택 필드. 파일 업로드를 처리할 때 사용	<input type="file">
hidden	숨겨진 입력 필드 – 미 노출 데이터를 전송할 때	<input type="hidden" name="token" value="12345">
range	슬라이더로 범위 값을 선택	<input type="range" min="0" max="100" step="10">

Attribute	Description
value	Specifies an initial value for an input field
readonly	Specifies that an input field is read-only.
disabled	Specifies that an input field should be disabled.
size	Specifies the visible width, in characters, of an input field.
maxlength	Specifies the maximum number of characters allowed in an input field.
min/max	Specifies the minimum and maximum values for an input field
multiple	Specifies that the user is allowed to enter more than one value in an input field.
pattern	Specifies a regular expression that the input field's value is checked against, when the form is submitted.
placeholder	Specifies a short hint that describes the expected value of an input field
required	Specifies that an input field must be filled out before submitting the form.
step	Specifies the legal number intervals for an input field
autofocus	Specifies that an input field should automatically get focus when the page loads
height/width	Specify the height and width of an <input type="image"> element.
list	Refers to a <datalist> element that contains pre-defined options for an <input> element.
autocomplete	Specifies whether a form or an input field should have autocomplete on or off.

- The <button> element defines a clickable button.
- Button Types – **type** attribute
 - type="button" - A normal clickable button (does nothing by default)
 - type="submit" - Submits a form
 - type="reset" - Resets all form fields
- Disabled Buttons – **disabled** attribute
 - Use the disabled attribute to make a button unclickable:

[실습] HTML Form Example I

HTML

```
<form>
  <fieldset>
    <legend>계정 정보</legend>
    <p>
      <label for="userId">아이디: </label>
      <input type="text" id="userId" name="userId" minlength="4" maxlength="15" required placeholder="4~15자 영문/숫자">
      <small>(필수)</small>
    </p>
    <p>
      <label for="userPw">비밀번호: </label>
      <input type="password" id="userPw" name="userPw" minlength="8" required placeholder="8자 이상 입력">
      <small>(필수)</small>
    </p>
    <p>
      <label for="userEmail">이메일: </label>
      <input type="text" id="userEmail" name="userEmail" placeholder="example">
      @
      <select name="emailDomain">
        <option value="direct">직접 입력</option>
        <option value="naver.com">naver.com</option>
        <option value="gmail.com">gmail.com</option>
        <option value="daum.net">daum.net</option>
      </select>
    </p>
  </fieldset>
</form>
```

계정 정보

아이디: (필수)

비밀번호: (필수)

이메일: @ ▼

[실습] HTML Form Example II

HTML

```
<form>
  <fieldset>
    <legend>개인 프로필 정보</legend>
    <p>
      <label for="userName">이름: </label>
      <input type="text" id="userName" name="userName" required>
    </p>
    <p>
      <label for="userBirth">생년월일: </label>
      <input type="date" id="userBirth" name="userBirth">
    </p>
    <p>
      성별:
      <label><input type="radio" name="gender" value="male" checked> 남성</label>
      <label><input type="radio" name="gender" value="female"> 여성</label>
      <label><input type="radio" name="gender" value="none"> 선택안함</label>
    </p>
    <p>
      관심 분야 (중복 선택 가능): <br>
      <label><input type="checkbox" name="interest" value="frontend"> 웹 프론트엔드 (Vue.js/HTML)</label><br>
      <label><input type="checkbox" name="interest" value="uiux"> UI/UX 디자인 표준</label><br>
      <label><input type="checkbox" name="interest" value="backend"> 백엔드 & 데이터베이스</label><br>
      <label><input type="checkbox" name="interest" value="devops"> 클라우드 & 인프라</label>
    </p>
  </fieldset>
</form>
```

개인 프로필 정보

이름:

생년월일:

성별: ☒ 남성 ☐ 여성 ☐ 선택안함

관심 분야 (중복 선택 가능):

- ☐ 웹 프론트엔드 (Vue.js/HTML)
- ☐ UI/UX 디자인 표준
- ☐ 백엔드 & 데이터베이스
- ☐ 클라우드 & 인프라

[실습] HTML Form Example III

HTML

```
<form>
  <fieldset>
    <legend>자기소개</legend>
    <p>
      <label for="intro">나를 표현하는 한 줄 소개 또는 가입 인사:</label><br>
      <textarea id="intro" name="intro" rows="5" cols="50" placeholder="여기에 내용을 입력하세요 (최대 200자)"></textarea>
    </p>
  </fieldset>
  <br>
  <input type="submit" value="동의하고 회원가입">
  <input type="reset" value="다시 작성">
</form>
```

자기소개

나를 표현하는 한 줄 소개 또는 가입 인사:

여기에 내용을 입력하세요 (최대 200자)

HTML Images -

- The HTML **** tag is used to embed an image in a web page.
- The **** tag has two required attributes
 - **src** - Specifies the path to the image
 - **alt** - Specifies an alternate text for the image
- 이미지의 크기를 지정할 때는 **width**, **height** 속성으로 지정하는 대신 **style** 속성을 통해 지정하는 것을 지향한다.

```
<body>  
  <h2>Images on Another Server</h2>  
    
</body>
```

Images on Another Server



HTML Images - <picture>

- The HTML **<picture>** element allows you to display different pictures for different devices or screen sizes.
 - The <picture> element contains one or more <source> elements.
 - Always specify an element as the last child element of the <picture> element.

```
<picture>  
  <source media="(min-width: 650px)" srcset="img_food.jpg">  
  <source media="(min-width: 465px)" srcset="img_car.jpg">  
    
</picture>
```

The picture Element



The picture Element



The picture Element



- The HTML **<audio>** element is used to play an audio file on a web page.
 - The **<source>** element allows you to specify alternative audio files which the browser may choose from. The browser will use the first recognized format.
 - The text between the **<audio>** and **</audio>** tags will only be displayed in browsers that do not support the **<audio>** element.
- Attributes
 - **controls**: play, pause, volume같은 기본 컨트롤 표시
 - **autoplay**: 자동 재생 (브라우저에서 제한될 수 있음)
 - **loop**: 반복 재생
 - **muted**: 음소거

```
<audio controls autoplay>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```



- The HTML **<video>** element is used to show a video on a web page.
 - The **<source>** element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.
 - The text between the **<video>** and **</video>** tags will only be displayed in browsers that do not support the **<video>** element.
- Attributes
 - **controls**: play, pause, volume같은 기본 컨트롤 표시
 - **autoplay**: 자동 재생 (브라우저에서 제한될 수 있음)
 - **poster**: 동영상 로딩 전 표시할 이미지
 - **width, height**: 비디오 플레이어의 크기 지정

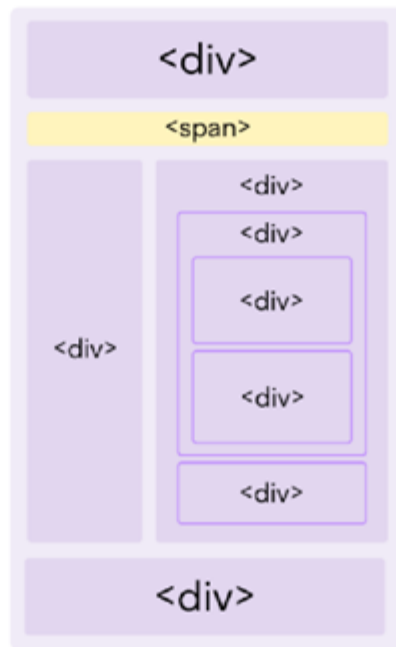
```
<video width="320" height="240" autoplay>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">  
  Your browser does not support the video tag.  
</video>
```



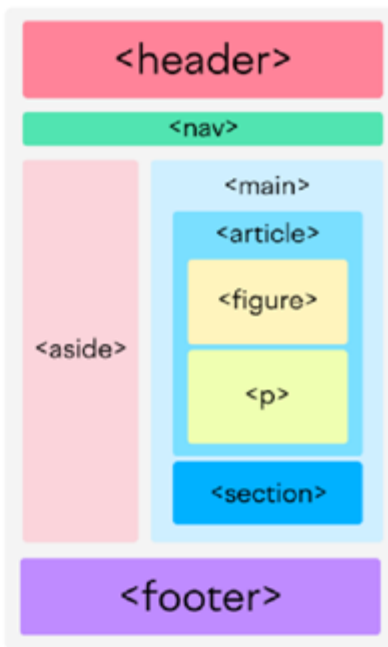
HTML Semantics - What are Semantic Elements?

- Semantic elements = elements with a meaning.
- A semantic element clearly describes its meaning to both the browser and the developer.
 - Examples of non-semantic elements: `<div>` and `<span>`
 - Examples of semantic elements: `<img>`, `<table>`, and `<article>`
- semantic element의 장점
 - 가독성: 코드의 구조가 명확하여 쉽게 이해
 - 접근성: 화면 낭독기 등의 보조 기술이 구조를 더 잘 이해
 - SEO(Search Engine Optimization)
: 검색 엔진이 콘텐츠의 구조와 주요 정보를 효율적으로 파악

Non-Semantic HTML



Semantic HTML



태그	설명
<header>	헤더 영역을 구분하는데 사용
<nav>	내부의 다른 영역이나 외부로 연결하는 링크 영역을 구분하는데 사용
<main>	문서의 주요 콘텐츠. 페이지에서 핵심적인 내용을 포함
<section>	논리적으로 관련 있는 내용 영역을 구분할 때 사용
<article>	독립적인 영역을 구분할 때 사용
<aside>	주력 내용이나 독립적인 내용으로 보기 어려워서 article 태그나 section 태그로 영역을 구분할 수 없을 때 사용
<footer>	footer 영역을 구분할 때 사용
<figure>	이미지, 도표, 코드 등 삽입된 콘텐츠를 감쌌. <figcaption>과 함께 사용
<figcaption>	<figure>에 대한 설명을 제공

- The `<head>` element is a container for metadata (data about data).
- The `<head>` element is placed between the `<html>` tag and the `<body>` tag
- HTML metadata is data about the HTML document. Metadata is not displayed on the page.
- The HTML `<head>` element is a container for the following elements
 - The `<title>` element is required and it defines the title of the document
 - The `<style>` element is used to define style information for a single document
 - The `<link>` tag is most often used to link to external style sheets
 - The `<meta>` element is typically used to specify the character set, page description, keywords, author of the document, and viewport settings
 - The `<script>` element is used to define client-side JavaScripts
 - The `<base>` element specifies the base URL and/or target for all relative URLs in a page

- Comments are not displayed by the browser, but they can help document your HTML source code.

```
<body>
  <!-- This is a comment -->
  <p>This is a paragraph.</p>
  <!-- Remember to add more information here -->
  <p>This <!-- great text --> is a paragraph.</p>
  <!--
  <p>Look at this cool image:</p>
  
  -->
  <p>This is a paragraph too.</p>
</body>
```

This is a paragraph.

This is a paragraph.

This is a paragraph too.

3. CSS

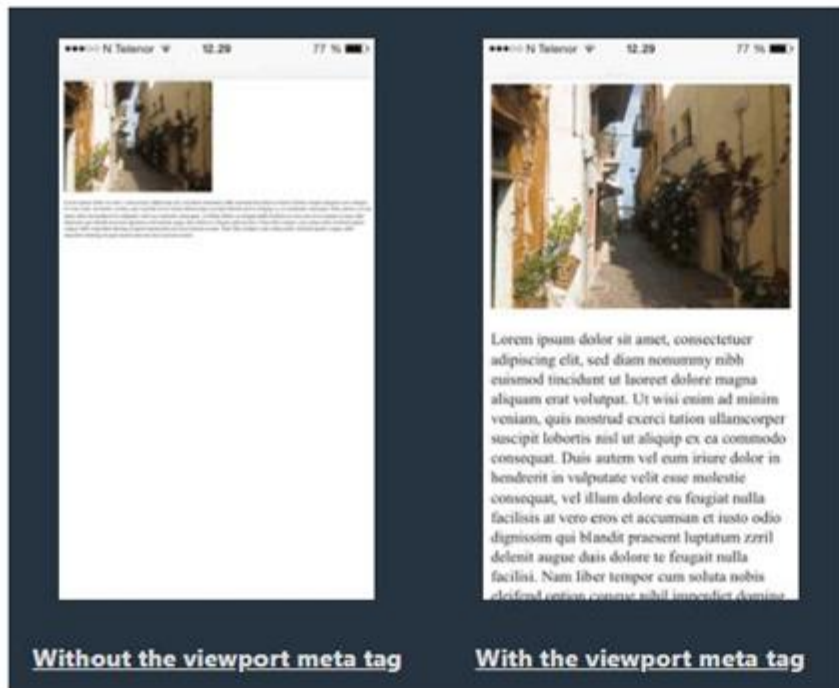
본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

- Cascading Style Sheets (CSS)는 웹 페이지의 스타일을 정의하는 스타일시트 언어
 - CSS stands for Cascading Style Sheets
 - CSS describes **how HTML elements are to be displayed** on screen, paper, or in other media
- CSS를 사용하면 색상, 글꼴, 텍스트 크기, element 간격, element 의 위치 및 레이아웃, 사용할 배경 이미지 또는 배경 색, 다양한 장치 및 화면 크기에 따른 표시 방식 등을 제어할 수 있습니다!
- CSS Demo - One HTML Page - Multiple Styles!
 - https://www.w3schools.com/css/css_intro.asp

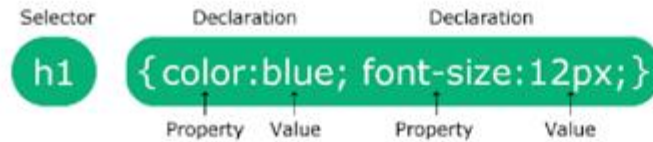
- 반응형 웹이 정상적으로 작동하려면 <head> 태그 안에 반드시 Viewport 메타 태그를 넣어주어야 한다.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- width=device-width: 기기의 실제 화면 너비에 맞춰 콘텐츠 폭을 설정
- initial-scale=1.0: 초기 확대/축소 비율을 1로 설정 (100%)



- A CSS rule consists of a **selector** and a **declaration block**



- The **selector** points to the HTML element you want to style.
- The **declaration block** contains **one or more declarations separated by semicolons**.
- Each declaration includes a CSS **property** name and a **value**, separated by a colon.
- Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

```
p {  
  color: red;  
  text-align: center;  
}
```

Hello World!

These paragraphs are styled with CSS.

- CSS를 사용하는 3가지 방법

- Inline CSS - by using the **style** attribute inside HTML elements

```
<h1 style="color:blue;text-align:center;">This is a heading</h1>  
<p style="color:red;">This is a paragraph.</p>
```

- Internal CSS - by using a **<style>** element in the **<head>** section

```
<html>  
<head>  
  <style>  
    body {background-color: linen;}  
    h1 {color: maroon; margin-left: 40px;}  
  </style>  
</head>  
<body>  
  <h1>This is a heading</h1>  
  <p>This is a paragraph.</p>  
</body>  
</html>
```

- CSS를 사용하는 3가지 방법
 - External CSS - by using a <link> element to link to an external CSS file

```
<html>
<head>
  <link rel="stylesheet" href="mystyle.css">
</head>
<body>
  <h1>This is a heading</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

```
body {background-color: lightblue;}
h1 {color: navy; margin-left: 20px;}
```

- CSS **selectors** are used to **"find" (or select) the HTML elements** you want to style.

- The CSS element Selector

```
p {  
  color: red;  
  text-align: center;  
}
```

- The CSS id Selector

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

- The CSS class Selector

```
.center {  
  text-align: center;  
  color: red;  
}
```

CSS Selectors - Grouping Selectors

- The universal selector (*) selects all HTML elements on the page.

```
* {  
  text-align: center;  
  color: blue;  
}
```

- The grouping selector selects all the HTML elements with the same style definitions.

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

- CSS **attribute selectors** are used to select and style **HTML elements with a specific attribute or attribute value, or both.**
 - [attribute] - Select elements with the specified attribute
 - [attribute="value"] - Select elements with a specific attribute and an exact value
 - [attribute~="value"] - Select elements with an attribute value containing a specific word
 - [attribute|= "value"] - Select elements with the specific attribute, whose value can be exactly the value, or start with the value followed by a hyphen (-)
 - [attribute^="value"] - Select elements whose attribute value starts with a specific value
 - [attribute\$="value"] - Select elements whose attribute value ends with a specific value
 - [attribute*="value"] - Select elements whose attribute value contains a specific value

```
input[type="text"] {  
  width: 150px;  
  padding: 6px;  
  margin-bottom: 10px;  
  background-color: pink;  
}  
  
input[type="button"] {  
  width: 100px;  
  padding: 6px;  
  background-color: lightgreen;  
}
```

- A CSS selector can contain more than one selector. Between the selectors, we can include a combinator, to create a more specific selection.
 - Descendant combinator (space): all elements that are **descendants (children, grandchildren, etc.)** of a specified element.
 - Child combinator (>) : all elements that are **direct children** of a specified element
 - Next sibling combinator (+) : an element that is **directly after a specific element**
 - Subsequent-sibling combinator (~) : all elements that are **next siblings** of a specified element

- CSS **selectors** are used to **"find" (or select)** the **HTML elements** you want to style.

선택자 종류	설명	예시
유니버설 선택자	모든 요소를 선택	<code>* { color: blue; }</code>
요소 선택자	특정 HTML 태그를 선택	<code>p { font-size: 16px; }</code>
아이디 선택자	특정 아이디를 가진 요소를 선택	<code>#header { background-color: grey; }</code>
클래스 선택자	특정 클래스를 가진 요소를 선택	<code>.menu { display: flex; }</code>
그룹화 선택자	여러 선택자를 그룹화하여 한 번에 스타일 적용	<code>h1, h2, h3 { color: green; }</code>
자식 선택자	직접 자식 요소를 선택	<code>div > p { margin-top: 10px; }</code>
후손 선택자	자식뿐 아니라 모든 후손 요소를 선택	<code>div p { color: red; }</code>
인접 형제 선택자	특정 요소 바로 다음에 위치한 형제 요소를 선택	<code>h1 + p { font-weight: bold; }</code>
일반 형제 선택자	특정 요소 뒤에 있는 모든 형제 요소를 선택	<code>h1 ~ p { color: blue; }</code>
속성 선택자	특정 속성을 가진 요소를 선택	<code>a[href] { color: red; }</code>
속성 값 선택자	특정 속성 값이 일치하는 요소를 선택	<code>a[href="https://example.com"] { font-weight: bold; }</code>

- A CSS pseudo-class is a keyword that can be added to a selector, **to define a style for a special state of an element.**
- Some common use for pseudo-classes
 - Style an element when a user moves the mouse over it
 - Style visited and unvisited links differently
 - Style an element when it gets focus
 - Style valid/invalid/required/optional form elements
 - Style an element that is the first child of its parent

```
selector:pseudo-class-name {  
  CSS properties  
}
```

- Interactive pseudo-classes apply styles **based on user interaction with elements**.
 - **:link** - Styles unvisited links
 - **:visited** - Styles visited links
 - **:hover** - Styles a link on mouse over
 - **:active** - Styles an activated link

- Structural pseudo-classes select elements **based on their position in the document tree**.
 - `:last-child` - Matches the last child of a parent
 - `:nth-child(n)` - Matches the nth child of a parent
 - `:nth-last-child(n)` - Matches the nth child from the end
 - `:only-child` - Matches an element that is the only child
 - `:first-of-type` - Matches the first element of its type
 - `:last-of-type` - Matches the last element of its type

- A CSS pseudo-element is a keyword that can be added to a selector, to **style a specific part of an element**.
- Some common use for pseudo-elements
 - Style the first letter or first line, of an element
 - Insert content before or after an element
 - Style the markers of list items
 - Style the user-selected portion of an element
 - Style the viewbox behind a dialog box

```
selector::pseudo-element-name {  
  CSS properties  
}
```

- Pseudo-elements for content allow you to insert content **before or after elements**, style list markers, and more.
 - **::before** - Insert some content before the content of a specified element.
 - **::after** - Insert some content after the content of a specified element.
 - **::marker** - style the list item markers.
 - **::selection** - style the part of a text that is selected by a user.
 - **::backdrop** - style the viewbox behind a dialog box or popover element.

- pseudo-elements for text, allow you to **style specific parts of text content, such as the first line or first letter.**
 - `::first-line` - Style the first letter or first line, of an element
 - `::first-letter` - content before or after an element

- Class가 notice인 p element

```
p.notice {font-size: 18px;}
```

- Id가 header인 div element

```
div#header {background: gray;}
```

- Class가 btn이면서 primary인 button element

```
button.btn.primary {background-color: blue;}
```

- Type attribute가 "text"이고 required attribute가 있는 input element

```
input[type="text"][required] {border: 1px solid red;}
```

- ul element 내 첫 번째 li element에 마우스를 올렸을 때

```
ul li:first-child:hover {color: red;}
```

- 마우스를 올린 모든 element

```
*:hover {background-color: yellow;}
```

- new class를 가진 section의 자식 중 featured class를 가진 article의 후손 중 title class를 가진 h2 element 바로 다음에 오는 summary class를 가진 p element

```
section.news > article.featured h2.title + p.summary {font-style: italic;}
```


CSS Code Challenge

- 1) CSS How To - https://www.w3schools.com/css/css_challenges_howto.asp
- 2) CSS Selectors - https://www.w3schools.com/css/css_challenges_selectors.asp
- 3) CSS Pseudo-classes - https://www.w3schools.com/css/css_challenges_pseudo_classes.asp

- CSS specificity is an algorithm that determines which style declaration is ultimately applied to an element.
- If two or more CSS rules point to the same element, the declaration with the highest specificity will "win", and that style will be applied to the HTML element.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p class="test">Hello World!</p>
</body>
</html>
```

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p id="demo" class="test">Hello World!</p>
</body>
</html>
```

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p id="demo" class="test" style="color:
pink;">Hello World!</p>
</body>
</html>
```

- Each type of CSS selector has a position in the specificity hierarchy, and the selector types carry different **"weights"**.

Selector	Example	Description	Weight
Inline styles	<h1 style="color: pink;">	1순위	
Id selectors	#navbar	2순위	1-0-0
Classes, attribute selectors and pseudo-classes	.test, [type="text"], :hover	3순위	0-1-0
Elements and pseudo-elements	h1, ::before, ::after	4순위	0-0-1
Universal selector and :where()	*, where()	우선순위 없음	0-0-0

- The selector with the higher leftmost number wins. If equal, compare the next number, and so on.

- CSS !Important Rule

- The !important rule is used to give the value of a specific property **the highest priority**.
- The !important rule will **override ALL previous styling rules** for that specific property on that element!
- The !important keyword is added to the end of a CSS declaration, before the semicolon.

```
selector {  
  property: value !important;  
}
```

- CSS !Important Rule 유의점 → 최소사용

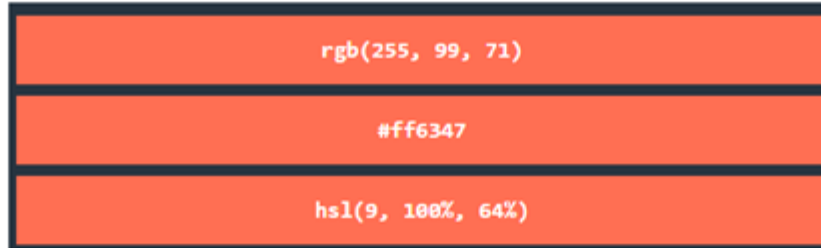
- 디버깅 어려움: 여러 곳에서 사용하면, CSS의 우선순위를 추적하기 어려워져 디버깅이 복잡해 짐.
- 유지 보수 어려움: 스타일 우선순위를 강제로 조정하므로, 코드 유지보수 및 확장성 측면에서 문제 가능
- 가독성 감소: !important를 남용하면 코드의 가독성이 떨어지고, 규칙 간의 관계를 명확하게 파악하기 어려움

- In CSS, colors are specified by using a predefined color name, or with a RGB, HEX, HSL, RGBA, HSLA value.

- Predefined color names



- RGB(Red, Green, Blue), HEX(Hexadecimal Color), HSL(Hue, Saturation, Lightness) values



- CSS units are used to define **the length and size of several properties**.
- Several CSS properties take "**length**" values, such as font-size, width, margin, padding, border, etc.
- Absolute Units
 - px (pixels) - The most used absolute unit for screens
- Relative Units
 - em - Relative to the font-size of the parent element
 - rem - Relative to the font-size of the root HTML element
 - % - Relative to the size of the parent element
 - vw - Relative to 1% of the width of the viewport*. 100vw = full width of the viewport
 - vh - Relative to 1% of the height of the viewport*. 100vh = full height of the viewport

```
body {font-size: 16px; /* Base font size */}
h1 {font-size: 2.5em; /* 2.5 * 16 = 40px */}
h2 {font-size: 1.875em; /* 1.875 * 16 = 30px */}
p {font-size: 1rem; /* 1 * 16 = 16px */}
```

* Viewport = the browser window size. 1vw = 1% of the current width of the browser's viewport. So, if the viewport is 500px wide, 1vw is 5px.

- Font Family – **font-family**

```
.p1 {font-family: "Times New Roman", Times, serif;}
```

- If the browser does not support the first font, it tries the next font. The font names should be separated with a comma.

- Font Style – **font-style, font-weight, font-variant**

```
p.italic {font-style: italic;}  
p.thick {font-weight: bold;}  
p.small {font-variant: small-caps;}
```

- Font Size – **font-size**

```
body {font-size: 16px; /* Base font size */}  
h1 {font-size: 2.5em; /* 2.5 * 16 = 40px */}  
h2 {font-size: 1.875rem; /* 1.875 * 16 = 30px */}  
p {font-size: 5vw;}
```

- Google Fonts are free to use, and have more than 1000 fonts to choose from
- How to use

```
<head>
  <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia">
  <style>
    body {
      font-family: "Sofia", sans-serif;
    }
  </style>
</head>
```

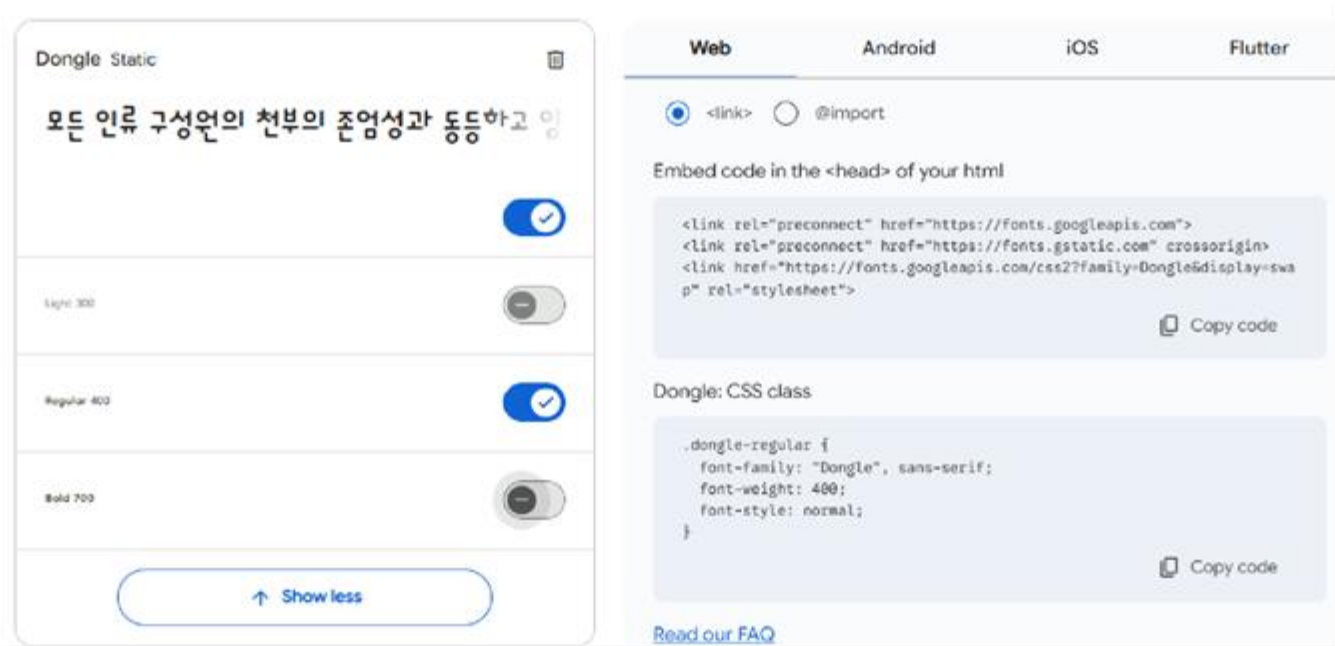
- Use multiple google fonts

```
<head>
  <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Audiowide|Sofia|Trirong">
  <style>
    h1.a {font-family: "Audiowide", sans-serif;}
    h1.b {font-family: "Sofia", sans-serif;}
    h1.c {font-family: "Trirong", serif;}
  </style>
</head>
```

- Enabling font effects – *effect=effectname*

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia&effect=fire">
```


- Google Fonts 공식 사이트에 접속 (<https://fonts.google.com/>)
- Language 필터에서 Korean (한국어) 선택 후 상세 페이지로 들어간다.
- 상세 페이지에서 Get font 이 후 장바구니에서 Get embed code 를 통해 확인한다.



- Text Color – **color**

```
body {color: blue;}  
h1 {color: #00ff00;}  
h2 {color: rgb(255,0,0);}
```

- Text Alignment – **text-align**

```
h1 {text-align: center;}  
h2 {text-align: left;}  
h3 {text-align: right;}
```

- Text Transformation – **text-transform**

```
p.uppercase {text-transform: uppercase;}  
p.lowercase {text-transform: lowercase;}  
p.capitalize {text-transform: capitalize;}
```

- Text Decoration – **text-decoration-line, text-decoration-color, text-decoration-style, text-decoration-thickness**

```
h1 {text-decoration-line: overline; text-decoration-color: red; text-decoration-style: double; text-decoration-thickness: 15px;}  
h3 {text-decoration-line: underline; text-decoration-color: green; text-decoration-style: dotted; text-decoration-thickness: 25%;}
```

- Text Spacing – **text-indent, letter-spacing, line-height, word-spacing, white-space**

```
p {text-indent: 50px; letter-spacing: 1px; line-height: 1.5; word-spacing: 5px; white-space: nowrap;}
```

- Text Shadow – **text-shadow**

```
h1 {text-shadow: 0 0 3px #ff0000, 0 0 5px #0000ff;}
```

- Background Color – **background-color, opacity**

```
body {background-color: lightblue; opacity: 0.3;}
```

- Background Image – **background-image**

```
body {background-image: url("paper.gif");}
```

- Background Image Repeat – **background-repeat, background-position**

```
body {background-image: url("img_tree.png"); background-repeat: no-repeat; background-position: right top;}
```

- Background Attachment – **background-attachment**

```
body {background-image: url("img_tree.png"); background-repeat: no-repeat; background-position: right top;  
background-attachment: scroll;}
```

- The CSS box model is essentially a box that wraps around every HTML element.

- Margin: 요소와 다른 요소 사이의 간격을 설정
- Padding: 콘텐츠와 경계 사이의 간격을 설정
- Border: 요소의 경계를 설정
- Content: 요소의 실제 내용이 들어가는 영역



- 브라우저 개발자도구에서 확인

- Elements > Computed



- width/height – width, height

```
div {height: 200px; width: 50%;}
```

- The height and width do not include padding, borders, or margins.
- 'auto' is default. The browser calculates the height and width.

- 너비/높이의 최소/최대 속성 – min-width, max-width, min-height, max-height

```
.div1 {max-width: 500px;}  
.div2 {width: 500px;}
```

- 브라우저의 창의 너비가 width보다 작으면 가로 스크롤바가 생기지만, max-width를 사용하면 스크롤바가 없이 너비가 맞춰서 줄어든다.

- Border Style – **border-style**

```
p.dotted {border-style: dotted;}
```

- 허용 값 : dotted, dashed, solid, double, groove, ridge, inset, outset, none, hidden

- Border Width – **border-width**

```
p.one {border-style: solid; border-width: 5px;}  
p.two {border-style: solid; border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */}  
p.three {border-style: solid; border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and 35px left */}
```

- Border Color – **border-color**

```
p.one {border-style: solid; border-color: red;}  
p.two {border-style: solid; border-color: red green blue yellow; /* red top, green right, blue bottom and yellow left */}
```

- Shorthand Border Property – **border, border-left, border-bottom, border-top, border-right**

```
p {border: 5px solid red;}  
p {border-left: 6px solid red;}
```

- Border-width, border-style, border-color를 한 번에 표시한다.

- Rounded Borders – **border-radius**

```
p {border: 2px solid red; border-radius: 5px;}
```

- Individual Margin – **margin-top, margin-right, margin-bottom, margin-left**

```
p {margin-top: 100px; margin-bottom: 100px; margin-right: 150px; margin-left: 80px;}
```

- Shorthand Margin Property – **margin**

```
p {margin: 25px 50px 75px 100px;}
```

- 순서대로 top, right, bottom, left의 값을 가진다.

- Margin Collapse (마진병합) 현상

- 여러 개의 세로 마진이 서로 만나면 각각 더해지는 것이 아니라, 그중 가장 큰 마진 값 하나로 합쳐지는 현상
- 좌우 마진은 병합되지 않는다.

```
h1 {margin-bottom: 50px;}  
h2 {margin-top: 20px;}
```

- Individual Padding – padding-top, padding-right, padding-bottom, padding-left

```
div {padding-top: 50px; padding-right: 30px; padding-bottom: 50px; padding-left: 80px;}
```

- Shorthand Padding Property – padding

```
div {padding: 25px 50px 75px 100px;}
```

- 순서대로 top, right, bottom, left의 값을 가진다.

- Element의 width와 padding

- Element의 width 속성은 element의 content 영역의 너비를 의미하므로, padding과 width가 같이 추가되어 있다면 실제 element의 총 너비는 두 값의 합과 같다.

```
div {width: 300px; padding: 25px; /* 총 너비는 300+25x2=350px */}
```


- **position** 속성은 element의 배치 유형을 지정하며 다음 값 중 하나를 가질 수 있다.
 - **static** – 기본값. element는 일반적인 문서 흐름에 따라 배치된다. (top, left 등 속성 값이 적용되지 않음)
`div.static {position: static; border: 3px solid #73AD21;}`
 - **Relative** – element는 문서 흐름 내의 일반적인 위치에 상대적으로 배치된다. (원래 공간은 차지함)
`div.relative {position: relative; left: 30px; border: 3px solid #73AD21;}`
 - **Fixed** – element는 뷰포트에 상대적으로 배치됩니다. (Scroll해도 위치가 고정된다.)
`div.fixed {position: fixed; bottom: 0; right: 0; width: 300px; border: 3px solid #73AD21;}`
 - **absolute** – element는 가장 가까운 조상 요소를 기준으로 배치된다. (문서 흐름에서 제거되고 공간은 차지하지 않는다.)
`div.absolute {position: absolute; top: 80px; right: 0; width: 200px; height: 100px; border: 3px solid red;}`
 - **sticky** - element는 스크롤 위치에 따라 relative와 fixed 사이를 전환한다. (조건부 고정으로 주로 헤더, 메뉴 등에서 사용된다.)
`div.sticky {position: sticky; top: 0; padding: 5px; background-color: #cae8ca; border: 2px solid #4CAF50;}`

- **z-index** 속성은 동일 위치에 중첩(overlap)되어 있는 element의 중첩 순서를 지정한다.
 - An element can have a positive or negative stack order (z-index):

```
<head>
  <style>
    img {
      position: absolute;
      left: 0px;
      top: 0px;
      z-index: -1;
    }
  </style>
</head>
<body>
  <h1>This is a heading</h1>
  
  <p>Because the image has a z-index of -1, it will be
placed behind the text.</p>
</body>
</html>
```



- 상속(Inheritance): 부모 요소에 지정된 스타일 중 일부가 자식 요소에 자동으로 적용되는 것
 - HTML 구조가 계층적(트리 구조)이기 때문에 CSS도 이런 계층적 스타일 적용이 가능
- 상속되는 속성

텍스트	color, text-align, text-indent, letter-spacing, word-spacing, line-height, visibility, white-space
폰트	font, font-family, font-size, font-style, font-variant, font-weight
리스트	list-style, list-style-type, list-style-position, list-style-image
테이블	border-collapse, border-spacing, caption-side, empty-cells

- 상속되지 않는 속성

레이아웃	margin, padding, border, width, height, display, position
배경	background-color, background-image
박스 정렬	flex, grid, justify-content, align-items
기타	box-shadow, z-index, overflow

- inherit 키워드를 사용하면 상속되지 않은 속성도 자식 요소로 강제로 상속이 가능하다.

```
<style>
p {
  border: 1px solid red;
}
strong {
  border: inherit;
}
</style>

<body>
<p>This is a paragraph with some <strong>strong</strong> text.</p>
</body>
```

CSS Code Challenge

- 1) CSS Colors - https://www.w3schools.com/css/css_challenges_colors.asp
- 2) CSS Fonts - https://www.w3schools.com/css/css_challenges_font.asp
- 3) CSS Text - https://www.w3schools.com/css/css_challenges_text.asp
- 4) CSS Backgrounds - https://www.w3schools.com/css/css_challenges_background.asp
- 5) CSS Position - https://www.w3schools.com/css/css_challenges_position.asp
- 6) CSS Inheritance - https://www.w3schools.com/css/css_challenges_inheritance.asp

▪ Layout 구성방식

방식	설명	장점	단점	사용 시기
float	float: left/right를 이용해 레이아웃 구성	간단한 구조에 적합	요소 겹침	구버전 브라우저 호환 목적
inline-block	display: inline-block으로 요소를 나란히 배치	float보다 자연스럽게 텍스트처럼 흐름에 따라 배치	여백(space) 문제 있음	간단한 레이아웃에 사용 가능
Flexbox	1차원(가로 또는 세로) 정렬에 특화된 현대적 레이아웃 방식	간단한 정렬/중앙정렬/비율 지정에 강력	2차원 배치에 제한	공간 내부 요소 정렬
Grid	2차원(행과 열) 레이아웃을 구성하는 최신 방식	복잡한 레이아웃도 간결하게 구성	1차원 구성이 복잡	페이지 전체 레이아웃 구성

- 일반적으로 Flexbox와 Grid를 함께 사용: 페이지 전체는 Grid, 내부 요소는 Flexbox로 정렬

- The **float** property specifies how an element should float within its container.
- The **float** property can have one of the following values:
 - **left** - The element floats to the left of its container
 - **right** - The element floats to the right of its container
 - **none** - Default. The element does not float and is displayed just where it occurs in the text
 - **inherit** - The element inherits the float value of its parent

```
<!DOCTYPE html>
<html>
<head>
<style>
div {float: left; padding: 15px;}
.div1 {background: red;}
.div2 {background: yellow;}
.div3 {background: green;}
</style>
</head>
<body>
<h2>Float Next To Each Other</h2>
<p>In this example, the three divs will float next to each other.</p>
<div class="div1">Div 1</div>
<div class="div2">Div 2</div>
<div class="div3">Div 3</div>
</body>
</html>
```

Float Next To Each Other

In this example, the three divs will float next to each other.



- 모든 element는 종류에 따라 가지고 있는 기본 display 값이 있는데 그것이 바로 Block과 Inline이다.
 - Block – 혼자서 한 줄 차지. <div>, <p>, <h1>~<h6>, ,
 - Inline – 줄 바꿈 없이 다른 요소와 같은 줄에 배치되어 width/height 속성이 무시됨. , <a>, ,
- display: inline-block은 배치는 Inline처럼 나란히, 성격은 Block처럼 크기 조절 가능하게 섞어 놓은 속성이다.

```
<head>
<style>
  .nav {background-color: lightgray; list-style-type: none; margin: 0; padding: 0;}
  .nav li {display: inline-block; font-size: 18px; padding: 15px;}
</style>
</head>
<body>
  <h1>Horizontal Navigation Menu</h1>
  <p>List items are displayed vertically by default. Here, we use display: inline-block to display them horizontally (side by side).</p>
  <ul class="nav">
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About Us</a></li>
    <li><a href="#clients">Our Clients</a></li>
    <li><a href="#contact">Contact Us</a></li>
  </ul>
</body>
</html>
```

Horizontal Navigation Menu

List items are displayed vertically by default. Here, we use display: inline-block to display them horizontally (side by side).

[Home](#) [About Us](#) [Our Clients](#) [Contact Us](#)

- 기존의 float나 inline-block 가지고 전체 레이아웃을 잡으려면 많이 복잡함.
- Flexbox는 가로 또는 세로 Layout을 효율적으로 구성할 수 있게 해 준다.
- Flexbox 핵심 개념

용어	설명
Flex Container	display: flex가 적용된 부모 요소
Flex Item	Flex Container의 직계 자식 요소들
Main Axis (메인 축)	Flex Item이 배치되는 중심 방향. Row (가로, 기본값) 또는 column (세로)
Cross Axis (교차 축)	Main Axis의 수직 방향

```
<style>
.container {
  display: flex;
  background-color: DodgerBlue;
}
.container div {
  background-color: #f1f1f1;
  margin: 10px;
  padding: 20px;
  font-size: 30px;
}
</style>
```

- The flex container element can have the following properties

속성	값	설명
flex-direction	row(default), column, row-reverse, column-reverse	아이템 배치 방향 지정
flex-wrap	nowrap(default), wrap, wrap-reverse	넘치는 아이템을 줄바꿈할지 여부
justify-content	center, flex-start(default), flex-end, space-around, space-between, space-evenly	주 축 정렬 방식
align-items	normal(default), stretch, center, flex-start, flex-end, baseline	교차 축에서 한 줄 정렬 방식
align-content	stretch(default), center, flex-start, flex-end, space-between, space-around 등	여러 줄 정렬 시 교차 축 처리

```
.container {
  display: flex;
  height: 200px;
  background-color: #f1f1f1;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
}
```

- Flex items can have the following properties
 - `order` - HTML 코드 순서와 상관없이 시각적인 배치 순서를 지정
 - `flex-grow` - 부모 박스 안에 남는 공간이 있을 때, 자식들이 그 공간을 얼마나 나눠 가질지(비율) 결정. 기본 값은 0
 - `flex-shrink` - 부모 박스가 자식들보다 작아질 때, 자식들이 얼마나 줄어들지 결정. 기본 값은 1
 - `flex-basis` - 자식 요소의 기본 크기를 설정 (`width`와 유사하지만 `flex` 환경에 더 최적화 됨)
 - `flex` - `flex-grow`, `flex-shrink`, and `flex-basis`를 한번에 쓰는 속성
 - `Align-self` - `flex container`가 정해진 정렬 규칙을 무시하고 이 아이템만 다르게 정렬하고 싶을 때 사용

```
.item {  
  flex-grow: 1;      /* 남은 공간 비율로 커짐 */  
  flex-shrink: 1;    /* 줄어드는 비율 */  
  flex-basis: 100px; /* 기본 크기 */  
  flex: 1 1 100px;   /* 위 3개의 축약형 */  
  align-self: center; /* 개별 정렬 */  
  order: 2;          /* 순서 변경 */  
}
```

- Flexbox가 1차원 레이아웃이라면 Grid는 2차원 레이아웃(행과 열)을 다룰 수 있다.
 - 페이지 전체 구조나 격자 기반 UI(갤러리, 대시보드 등)에 적합
- Grid 핵심 개념

용어	설명
Grid Container	display: grid가 선언된 부모 요소
Grid Item	Grid Container의 직계 자식 요소
Grid Line	행과 열을 나누는 선
Grid Track	행(row) 또는 열(column)
Grid Cell	하나의 셀 (row와 column이 만나는 공간)
Grid Area	여러 셀을 묶은 영역

My Header

[Link 1](#)
[Link 2](#)
[Link 3](#)

Lorem Ipsum

Lorem ipsum odor amet, consectetur adipiscing elit. Ridiculus sit nisl laoreet facilisis aliquet. Potenti dignissim litora eget montes rhoncus sapien neque urna. Cursus libero sapien integer magnis ligula lobortis quam ut.

Footer

Grid Container의 속성

구분	속성	예시 값	설명
Grid Tracks	grid-template-columns	200px 1fr 1fr	열의 수와 너비 정의
	grid-template-rows	100px auto	행의 수와 높이 정의
Grid Gaps	gap, row-gap, column-gap	10px	행과 열 사이 간격 설정
Grid Align	justify-items	start, end, center, stretch	각 셀 내 요소의 수평 정렬
	align-items	start, end, center, stretch	각 셀 내 요소의 수직 정렬

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr; /* 열 정의 */
  grid-template-rows: auto 100px; /* 행 정의 */
  gap: 10px; /* 셀 사이 간격 */
  justify-items: center; /* 셀 내부 가로 정렬 */
  align-items: center; /* 셀 내부 세로 정렬 */
}
```

- Grid items can have the following properties
 - 그리드 라인 번호로 위치 지정하기 : grid-column-start, grid-column-end, grid-column, grid-row-start, grid-row-end, grid-row
(참고로, 그리드는 셀(칸)의 개수가 아니라, 칸을 나누는 선(Grid Line)의 번호를 기준으로 구역을 잡는다.)
 - Grid Container가 기정한 이름에 배치하기 : grid-area
 - Grid Container 정렬규칙을 따르지 않고 개별 정렬하기 : justify-self, align-self

```
.item {  
  grid-column: 1 / 3;    /* 1번 선부터 3번 선까지 열 병합 */  
  grid-row: 2 / 4;       /* 2번 선부터 4번 선까지 행 병합 */  
  justify-self: center; /* 개별 요소 수평 정렬 */  
  align-self: end;       /* 개별 요소 수직 정렬 */  
}
```

CSS Code Challenge

- 1) CSS Float - https://www.w3schools.com/css/css_challenges_float.asp
- 2) CSS Inline-block - https://www.w3schools.com/css/css_challenges_inline-block.asp
- 3) Flexbox Intro - https://www.w3schools.com/css/css_challenges_flexbox_intro.asp
- 4) Flexbox Container - https://www.w3schools.com/css/css_challenges_flexbox_container.asp
- 5) Flexbox Items - https://www.w3schools.com/css/css3_flexbox_items.asp
- 6) Grid Intro - https://www.w3schools.com/css/css_challenges_grid_intro.asp
- 7) Grid Container - https://www.w3schools.com/css/css_challenges_grid_container.asp
- 8) Grid Items - https://www.w3schools.com/css/css_challenges_grid_item.asp

- The CSS **transform** property applies a 2D or 3D transformation to an element.
 - transform 속성은 element를 회전하거나, 크기조정을 하거나, 이동시키거나, 기울이는데 사용한다.
- Transform Functions: transform 속성이 사용하는 함수
- 2D Transform 함수 사용 예

```
transform: rotate(45deg);    /* 45도 회전 */  
transform: scale(1.2);       /* 크기 1.2배 */  
transform: translateX(50px); /* X축으로 50px 이동 */
```


- The CSS **transform** property applies a 2D or 3D transformation to an element.
 - rotateX(), rotateY(), rotateZ() 함수를 통해 X축, Y축, Z축을 기준으로 element를 회전할 수 있다.
- Transform Functions: transform 속성이 사용하는 함수
- 3D Transform 속성 목록

Property	Description
transform	Applies a 2D or 3D transformation to an element
transform-origin	Allows you to change the position on transformed elements
transform-style	Specifies how nested elements are rendered in 3D space
perspective	Specifies the perspective on how 3D elements are viewed
perspective-origin	Specifies the bottom position of 3D elements
backface-visibility	Defines whether or not an element should be visible when not facing the screen

- CSS의 속성에 대한 값이 변할 때, 변화 시의 효과를 설정한다.

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background-color: red;
  transition: width 2s, height 4s, background-color 3s;
  /* width 변하는데 2초, height 변하는데 4초, 배경색 변하는데 3초 걸림 */
}
div:hover {
  width: 300px;
  height: 300px;
  background-color: orange;
}
</style>
</head>
<body>
  <h1>The transition Property</h1>
  <p>Hover over the div element below, to see the transition effect:</p>
  <div></div>
</body>
</html>
```

- 전환 효과와 관련된 속성
 - `transition-timing-function`: transition 효과가 진행되는 속도의 형태 지정
 - `transition-delay`: transition이 시작되기 전의 지연 시간 지정
 - `transition-duration`: transition 효과가 진행되는 시간 지정
 - `transition-property`: transition 효과를 부여할 css property 지정

```
div {  
  transition-property: width;  
  transition-duration: 2s;  
  transition-timing-function: linear;  
  transition-delay: 1s;  
}
```

- `transition`: 위 4개의 transition 속성을 한번에 작성할 수 있는 속성 (shorthand property)

```
div {  
  transition: width 2s linear 1s;  
}
```

- CSS Transition은 element의 전후(시작과 종료) 상태를 부드럽게 변화한다.
- CSS Animation은 시작과 종료 뿐만 아니라 중간 상태도 고려하여 더 복잡하고 다양한 효과를 제공한다.

속성	의미	기본값
animation-name	@keyframes의 이름	none
animation-duration	지속 시간	0s
animation-delay	대기 시간	0s
animation-timing-function	타이밍 함수	ease
animation-iteration-count	반복 횟수	1
animation-direction	반복 방향	normal
animation-fill-mode	전후 상태	none
animation-play-state	재생과 정지	running

- @keyframe은 애니메이션의 각 단계를 설정한다.

- 반응형 웹 디자인(Responsive Web Design)은 하나의 **HTML** 소스로 **PC**, 태블릿, 스마트폰 등 다양한 크기의 기기(화면)에 맞춰 레이아웃이 자동으로 변형되도록 디자인하는 웹 개발 방식을 말한다.
- 반응형 웹의 핵심 기술
 - Viewport <meta> tag
 - Mediaqueries
 - Flexible Layout (Grid and Flex)
- 브라우저는 웹 페이지를 렌더링할 때 기기의 화면 정보를 파악
 - width, height: 디바이스 또는 브라우저의 뷰포트 크기
 - orientation: 세로(portrait) 또는 가로(landscape)
 - resolution: 해상도 (dpi 또는 dppx)
 - color, aspect-ratio 등도 감지 가능



- 디바이스의 화면 크기, 방향, 해상도 등 미디어 특성에 따라 서로 다른 스타일을 적용할 수 있게 해주는 핵심 기술
 - 화면 레이아웃이 바뀌는 기준이 되는 지점을 브레이크포인트(Breakpoint)라고 부른다.
- Breakpoints 샘플

```
/* 1. 모바일 (Mobile) */ /* 기본 스타일은 모바일 기준으로 작성 (Mobile First 방식 시) */
@media (max-width: 767px) {
  /* 스마트폰 가로/세로, 대화면 스마트폰 */
}

/* 2. 태블릿 (Tablet) */
@media (min-width: 768px) and (max-width: 1023px) {
  /* 아이패드 등 일반적인 태블릿 디바이스 */
}

/* 3. 소형 노트북 / PC (Laptop) */
@media (min-width: 1024px) and (max-width: 1439px) {
  /* 13인치, 15인치 노트북 및 작은 모니터 해상도 */
}

/* 4. 대화면 데스크톱 (Desktop) */
@media (min-width: 1440px) {
  /* 일반적인 모니터(1920px 이상) 및 대화면 디스플레이 */
}
```

- 많은 웹 페이지는 그리드 뷰를 기반으로 하는데, 반응형 그리드 뷰는 일반적으로 6개 또는 12개의 열을 가진다.
- Grid View 샘플 : https://www.w3schools.com/css/css_rwd_grid.asp

- 반응형을 짤 때 두 가지 접근 방식
 - Mobile First: 모바일 화면 스타일을 먼저 만들고, 미디어 쿼리(min-width)를 사용해 화면이 커질 때마다 살을 붙여 나가는 방식
 - Desktop First: PC 화면을 먼저 만들고, 미디어 쿼리(max-width)를 사용해 화면이 작아질 때마다 요소를 떼어내거나 줄이는 방식
- 사용자들이 모바일 기기를 우선적으로 사용하는 시대 흐름에 맞춰 Mobile-First Design이 권장됨

항목	설명
설계 순서	모바일 → 태블릿 → 데스크탑
장점	핵심 기능에 집중, 로딩 속도 최적화, 반응형 구현에 유리
기술적 구현	CSS min-width 기반 미디어 쿼리 사용

- 변수 선언하기: --변수명

- 보통 CSS 변수는 사이트 전체에서 전역적으로 쓰기 위해 가장 상위 요소인 :root에 선언한다.
- :root는 HTML 문서의 가장 최상단 요소(<html> 태그)를 가리키는 가상 선택자이다.

```
:root {  
  /* --변수명: 값; 구조로 선언합니다 */  
  --main-color: #007bff;  
  --danger-color: #dc3545;  
  --default-padding: 16px;  
}
```

- 변수 사용하기: var(--변수명)

```
.button-primary {  
  background-color: var(--main-color); /* #007bff가 적용됨 */  
  padding: var(--default-padding);    /* 16px이 적용됨 */  
}  
.  
alert-box {  
  border: 2px solid var(--danger-color);  
}
```

- CSS 변수 사용의 장점

- 반복되는 값의 재사용이 가능하다.
- 특히, JavaScript에서 동적으로 값을 변경하여, 테마(다크/라이트 모드) 전환 기능을 구현할 때 핵심 기술로 사용된다.

■ CSS Vs. SCSS/SASS

구분	CSS	SCSS (Sassy CSS)	SASS (Syntactically Awesome Style Sheets)
정의	웹 표준 스타일시트 언어	SASS의 3세대 문법 (CSS 친화적)	CSS를 확장한 전처리기 언어
구문 형태	중괄호({})와 세미콜론(;)	중괄호({})와 세미콜론(;)	들여쓰기와 줄바꿈
가독성	구조가 커질수록 복잡해짐	CSS와 유사하여 가독성 좋음	중괄호가 없어 깔끔하지만 호불호
호환성	브라우저가 직접 인식함	CSS와 100% 호환됨	전통적인 SASS 문법 (적용 필요)
사용 빈도	기본 필수	현재 업계 표준/가장 많이 씀	점차 줄어드는 추세

```
nav {
  background-color: #333;
}
nav ul {
  list-style: none;
}
nav ul li {
  display: inline-block;
}
```

```
nav {
  background-color: #333;
  ul {
    list-style: none;
    li {
      display: inline-block;
    }
  }
}
```

```
nav
  background-color: #333
  ul
    list-style: none
    li
      display: inline-block
```

- SCSS/SASS 장점
 - 변수 사용 (\$variable): 색상, 폰트 크기 등을 변수로 지정해 두고 재사용할 수 있다. (예: \$primary-color: #ff0000;)
 - 중첩 (Nesting): 상위 선택자의 반복을 줄여 컴포넌트 단위로 관리가 편해진다.
 - 믹스인 (Mixins): 자주 사용하는 CSS 코드 블록을 정의해 두고 필요할 때마다 불러올 수 있다. (함수와 유사)
 - 모듈화 (Import): 스타일시트를 여러 파일로 쪼개어 관리하고 하나로 합칠 수 있어 **대규모 프로젝트에 필수적이다**.
- Convert SCSS/SASS code into standard CSS
 - SCSS나 SASS로 작성한 코드는 브라우저가 직접 읽을 수 없기 때문에, 작성된 스타일시트를 일반 CSS로 바꾸어주는 과정이 필요
 - 이 과정을 트랜스파일링(Transpiling)이라 한다.
 - Vite, Webpack, Turbopack 같은 Frontend Build 도구에서는 설정을 통해 자동으로 변환시킨다.

CSS Code Challenge

- 1) 2D Transforms - https://www.w3schools.com/css/css_challenges_css3_transforms.asp
- 2) 3D Transforms - https://www.w3schools.com/css/css_challenges_css3_3dtransforms.asp
- 3) Transition - https://www.w3schools.com/css/css_challenges_css3_transitions.asp
- 4) Animations - https://www.w3schools.com/css/css_challenges_css3_animations.asp
- 5) RWD Intro - https://www.w3schools.com/css/css_challenges_rwd_intro.asp
- 6) RWD Viewport - https://www.w3schools.com/css/css_challenges_rwd_viewport.asp
- 7) RWD Grid View - https://www.w3schools.com/css/css_challenges_rwd_grid.asp
- 8) RWD Media Queries - https://www.w3schools.com/css/css_challenges_rwd_grid.asp
- 9) CSS Variables - https://www.w3schools.com/css/css_challenges_css3_variables.asp

4. JavaScript

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

- 웹 브라우저와 서버 모두에서 작동하는 프로그래밍 언어
- 인터랙티브한 웹 페이지와 다양한 애플리케이션을 개발하기 위해 사용
- HTML과 CSS와 함께 웹 개발의 핵심 기술 중 하나로 간주
 - JavaScript Can Change HTML Content
 - JavaScript Can Change HTML Attribute Values
 - JavaScript Can Change HTML Styles (CSS)
 - JavaScript Can Hide/Show HTML Elements

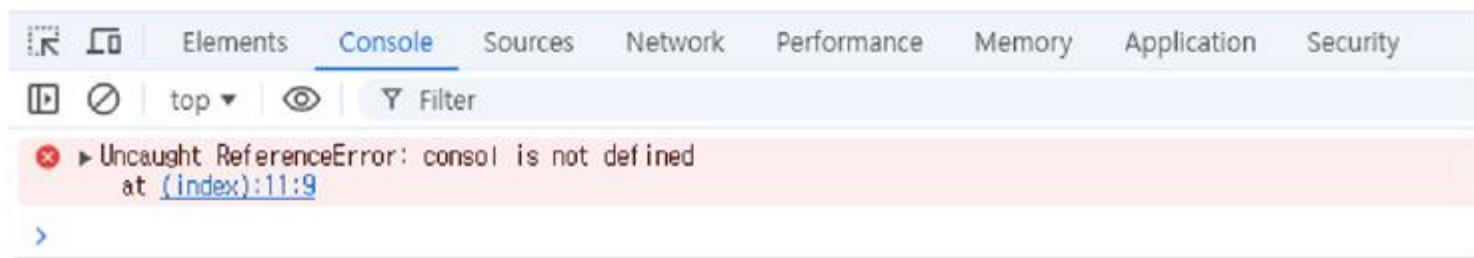
- HTML문서에서 JavaScript 코드는 `<script>` `</script>` Tag 사이에 작성되어야 한다.

```
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

- JavaScript는 `<body>`나 `<head>` section 내에서 작성된다.
 - `<head>`에 스크립트를 넣으면 스크립트가 로드되는 동안 HTML 요소가 렌더링되지 않기 때문에 페이지가 늦게 표시된다.
 - `<body>` 끝에 스크립트를 넣으면 HTML 문서의 모든 콘텐츠가 먼저 렌더링 된 후 실행됨으로 사용자에게 더 빠른 경험을 제공
- External JavaScript

```
<script src="https://www.w3schools.com/js/myScript.js"></script>
<script src="/js/myScript.js"></script>
<script src="myScript.js"></script>
```

- JavaScript가 데이터를 보여주는 방법
 - Writing into an HTML element, using innerHTML or innerText.
 - Writing into the HTML output using document.write().
 - Writing into an alert box, using window.alert().
 - Writing into the browser console, using console.log().
- 개발자 도구 Console Tab
 - 단축키 : Ctrl + Shift + I
 - Console Tab에서 console.log()를 확인할 수 있다.
 - Console Tab에서 자바스크립트 에러를 확인할 수 있다.



- JavaScript Value는 Literals(Fixed Values)와 Variables(Variable Values) 2종이 존재한다.
- JavaScript Keyword는 수행할 동작을 정의하는 데 사용된다.
- JavaScript Variable은 데이터를 저장하기 위해 이름을 붙인 저장 공간이다.
- JavaScript Identifier는 변수 이름이다.
- JavaScript Operator는 변수간 연산을 하는 연산자이다.
- JavaScript Expression은 value, variables, operators의 조합이다.
- JavaScript는 Case-Sensitive 하다.

- JavaScript Programs
 - A computer program is a list of "instructions" to be "executed" by a computer.
 - These programming instructions are called statements.
 - The statements are executed, one by one, in the same order as they are written.
 - In HTML, JavaScript programs are executed by the web browser.
- JavaScript Statements
 - JavaScript statements are composed of Values, Operators, Expressions, Keywords, and Comments.
 - Semicolons separate JavaScript statements.
 - JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.
 - JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.
 - JavaScript keywords are reserved words. Reserved words cannot be used as names for variables.
- JavaScript Comments
 - Single line comments start with `//`.
 - Multi-line comments start with `/*` and end with `*/`.

- JavaScript variables are containers for data.
 - Variables are identified with unique names called identifiers.
- Declaring JavaScript Variables
 - Declaring a Variable Using let
 - Declaring a Variable Using const
 - Declaring a Variable Automatically (Not Recommended)
 - Declaring a Variable Using var (Not Recommended)

Keyword	범위(Scope) = 변수 접근 가능 범위	재선언	재할당
var	함수 범위(function scope)	O	O
let	블록 범위(block scope)	X	O
const	블록 범위(block scope), 상수	X	X

- JavaScript variables have 3 types of scope
 - Global Scope: Global variables can be accessed from anywhere in a JavaScript program.

```
let carName = "Volvo";  
// code here can use carName  
  
function myFunction() {  
  // code here can also use carName  
}
```

- Block Scope: Variables declared inside a { } block cannot be accessed from outside the block (let, const 선언에 해당)

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

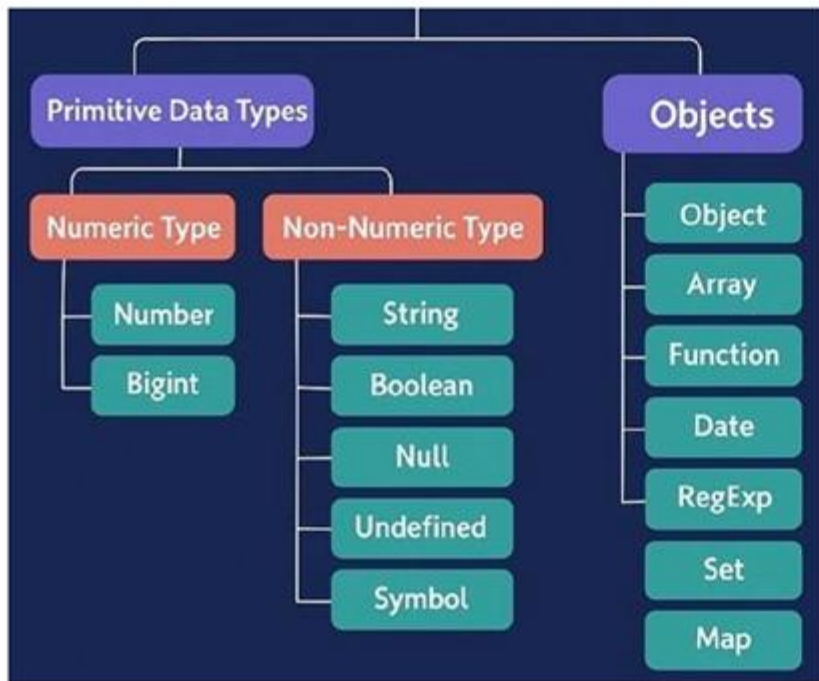
- Function Scope: Inside a function all variables declared with var, let or const have Function Scope

```
function myfunction() {  
  var x = 1;  
  let y = 2;  
  const z = 3;  
}  
//x can NOT be used here  
//y can NOT be used here  
//z can NOT be used here
```

- Declaring a Variable Using **let**
 - The let keyword was introduced in ES6 (2015)
 - Variables declared with let have Block Scope
 - Variables declared with let must be Declared before use
 - Variables declared with let cannot be Redeclared in the same scope

- Declaring a Variable Using **const**
 - The const keyword was introduced in ES6 (2015)
 - Variables defined with const cannot be Redeclared
 - Variables defined with const cannot be Reassigned
 - Variables defined with const have Block Scope
- Constant Objects and Arrays
 - It does not define a constant value. It defines a constant reference to a value.

- JavaScript Datatypes: 7개 기본형(Primitive Data Types) + 1개 참조형(Object Data Type, Reference Data Type)으로 구성
- JavaScript has dynamic types. This means that the same variable can be used to hold different data types:



```
let x;      // Now x is undefined
x = 5;      // Now x is a Number
x = "John"; // Now x is a String
```

- JavaScript Strings
 - **Using Quotes:** A JavaScript string is zero or more characters written inside quotes.
 - **Template Strings:** Templates are strings enclosed in backticks
 - **String Length:** To find the length of a string, use the built-in length property
 - **Escape Characters:** The backslash escape character (\) turns special characters into string characters:
 - **JavaScript Strings as Objects:** Normally, JavaScript strings are primitive values, created from literals, but strings can also be defined as objects with the keyword new
- JavaScript String Templates (=Template Strings, Template Literals)
 - **Back-Tics Syntax:** Template Strings use back-ticks (` `) rather than the quotes ("") to define a string
 - **Multiline Strings:** Template Strings allow multiline strings
 - **Interpolation:** Template strings provide an easy way to interpolate variables in strings.
 - **Expression Substitution:** Template Strings allow interpolation of expressions in strings:

- Basic String Methods

- Javascript strings are primitive and **immutable**: All string methods produce a new string without altering the original string.
- String.concat(), String.substring(), String.substr(), String.toUpperCase(), String.trim(), String.replace(), String.split()...

- String Search Methods

- String.indexOf(), String.lastIndexOf(), String.search(), String.match(), String.matchAll(), String.includes(), String.startsWith()...

- Wrapper Object

- 자바스크립트 엔진은 원시 타입의 속성이나 메서드에 접근하려고 할 때, 순간적으로 그 원시 타입을 연관된 객체로 임시 변환한다.

```
const name = "gemini"; // 원시 타입인 string
console.log(name.toUpperCase()); // "GEMINI"가 정상 출력됨! 객체인가..?
```

- typeof 연산자를 통한 확인

```
const primitiveStr = "hello";
const objectStr = new String("hello"); // 명시적으로 객체로 만들

console.log(typeof primitiveStr); // "string" (원시 타입)
console.log(typeof objectStr);    // "object" (객체)
```

- Do not create String objects. The new keyword complicates the code and slows down execution speed.

- JavaScript Numbers
 - **Number Types:** Unlike many other programming languages, JavaScript does not define different types of numbers, like integers, short, long, floating-point etc
 - **Integer Precision :** Integers (numbers without a period or exponent notation) are accurate up to 15 digits.
 - **Floating Precision :** Floating point arithmetic is not always 100% accurate
 - **Adding Numbers and Strings :** JavaScript uses the + operator for both addition and concatenation. Numbers are added. Strings are concatenated.
 - **NaN(Not a Number):** NaN is a JavaScript reserved word indicating that a number is not a legal number.
 - **Infinity:** Infinity (or -Infinity) is the value JavaScript will return if you calculate a number outside the largest possible number. Division by 0 (zero) also generates Infinity
 - **Hexadecimal:** JavaScript interprets numeric constants as hexadecimal if they are preceded by 0x.
 - **JavaScript Numbers as Objects:** Normally JavaScript numbers are primitive values created from literals, But numbers can also be defined as objects with the keyword new:

JS Datatypes - Number Methods and Properties

JavaScript

- Basic Number Methods
 - **toString()**, toExponential(), toFixed(), toPrecision(), **valueOf()**
- Converting Variables to Numbers
 - Number(), **parseInt()**, **parseFloat()**
- Number Object Methods
 - Number.isInteger(), Number.isNaN(), Number.isFinite(), Number.isSafeInteger(), Number.parseInt(), Number.parseFloat()
- Number Object Properties
 - Number.EPSILON, Number.MAX_VALUE, Number.MIN_VALUE, Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER, Number.POSITIVE_INFINITY, Number.NEGATIVE_INFINITY, Number.NaN

- What is JavaScript BigInt?
 - Number의 범위를 초과하는 정수를 표현하기 위해 사용 (Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER).
- How to Create a BigInt
 - Using an integer literal with an n suffix
 - Using the BigInt() constructor with a string

```
let x = 12345678901234567890n;  
let y = BigInt("12345678901234567890")
```

- Arithmetic between a BigInt and a Number is not allowed (will result in a TypeError).

```
let x = 10n;  
let y = 5;  
  
let z = x + y; // ✖ TypeError
```

- Key Boolean Characteristics
 - **true** and **false** are the only possible boolean values
 - true and false must be written in lowercase
 - true and false must be written without quotes

- 비교 연산의 결과는 true/false를 return한다.

```
let boolean1 = 10 < 20;  
let boolean2 = 10 > 20;  
console.log(boolean1); // true  
console.log(boolean2); // false
```

- 조건문과 반복문에 광범위하게 사용된다.
- Everything With a "Value" is True
- Everything Without a "Value" is False

JS Datatypes - undefined, null

JavaScript

- Undefined

- 변수 선언은 했지만 값이 아직 할당되지 않았을 때, undefined 타입

```
let something;  
console.log(something); // undefined
```

- Accessing a non-existing object property returns undefined.

```
const person = {firstName:"John", lastName:"Doe"};  
document.getElementById("demo").innerHTML = person.age;
```

- A function without a return value returns undefined.

```
function myFunction() {  
  let x = 5;  
}  
document.getElementById("demo").innerHTML = myFunction();
```

- null

- null 은 "값이 없음"이나 "의도적으로 비어 있음"을 표현, null 타입

```
let nothing = null;  
console.log(nothing); // null
```

- JavaScript Symbols

- 고유하고 변경 불가능한 값으로, 주로 객체의 고유한 속성 키로 사용
- Symbol() 함수를 사용해 생성하며, 같은 입력값을 넣어도 결과값은 달라짐
- 문자열로 자동 변환되지 않으며 + 연산도 불가능

```
let sym1 = Symbol("id");  
let sym2 = Symbol("id");  
console.log(sym1 === sym2); // false
```

▪ Reference Data Types

- JavaScript의 참조형(Reference) 데이터 타입은 데이터의 실제 값이 아닌, 그 값이 저장된 메모리 주소(참조값)를 변수에 저장하는 데이터 타입이다.
- Heap Memory 저장: 크기가 정해지지 않은 복잡한 데이터이기 때문에, 실제 데이터는 메모리의 '힙'이라는 공간에 저장되고, 변수에는 그 위치를 가리키는 '주소'만 복사된다.
- 가변성(Mutable): 기본형과 달리, 값이 복사되는 것이 아니라 주소가 공유되기 때문에 객체의 내부 프로퍼티를 변경하면 해당 주소를 참조하는 다른 변수의 값도 같이 변경된다

자료형	설명	예시
object	키-값 쌍으로 이루어진 데이터 구조	<code>const user = { name: "홍길동", age: 25 };</code>
array	순서가 있는 값들의 집합	<code>let items = [1, 2, 3];</code>
function	실행 가능한 코드 블록	<code>const greet = function() { ... };</code>
date	날짜와 시간 객체	<code>let today = new Date();</code>
map	키-값 쌍을 저장하는 컬렉션 (key는 다양한 타입 가능)	<code>const map = new Map();</code>
set	중복되지 않는 값의 집합	<code>const set = new Set([1, 2, 3]);</code>
regexp	정규 표현식	<code>const regex = /abc/i;</code>

- Types of JavaScript Operators
 - Arithmetic Operators
 - Assignment Operators
 - Comparison Operators
 - Logical Operators
 - And more ...

연산자	설명	예제	결과
+	덧셈	5 + 3	8
-	뺄셈	5 - 3	2
*	곱셈	5 * 3	15
/	나눗셈	6 / 2	2
%	나머지	5 % 2	1
**	거듭제곱	2 ** 3	8
++	1 증가	let a = 1; a++; ++a;	a = 2
--	1 감소	let a = 1; a--; --a;	a = 0

JS Operators - Assignment

JavaScript

연산자	설명	예제	결과
=	값 할당	let a = 5	a = 5
+=	덧셈 후 할당	a += 2	a = 7
-=	뺄셈 후 할당	a -= 2	a = 3
*=	곱셈 후 할당	a *= 2	a = 10
/=	나눗셈 후 할당	a /= 2	a = 2.5
%=	나머지 후 할당	a %= 2	a = 1

JS Operators - Comparisons

JavaScript

연산자	설명	예제	결과
==	[느슨한 비교] 값이 같으면 true	5 == '5'	TRUE
===	[엄격한 비교] 값과 타입이 같으면 true	5 === '5'	FALSE
!=	[느슨한 비교] 값이 다르면 true	5 != '5'	FALSE
!==	[엄격한 비교] 값 또는 타입이 다르면 true	5 !== '5'	TRUE
<, >, <=, >=	크기 비교	5 > 3	TRUE

JS Operators - Logical

JavaScript

연산자	설명	예제	결과
&&	논리 AND	true && false	FALSE
	논리 OR	true false	TRUE
!	논리 NOT	!true	FALSE

연산자	설명	예제	결과
&	비트 AND	5 & 3	1
	OR	비트 OR	`5
^	비트 XOR	5 ^ 3	6
~	비트 NOT	~5	-6
<<, >>, >>>	비트 시프트	5 << 1	10

연산자	설명	예제	결과
삼항 연산자 (? :)	조건부 연산자 (삼항 연산자)	let result = (5 > 3) ? 'Yes' : 'No'	Yes
typeof 연산자	데이터 타입 반환	typeof 42	'number'
in 연산자	속성 여부 확인	'length' in [1, 2, 3]	TRUE
instanceof 연산자	객체의 인스턴스 여부 확인	[] instanceof Array	TRUE

- The JavaScript if Statement

- Use the JavaScript if statement to execute a block of code **when a condition is true**.

```
let age = 16;  
let country = "USA";  
let text = "You can Not drive!";  
  
if (country == "USA" && age >= 16) {  
  text = "You can drive!";  
}
```


- The JavaScript else Statement

- Use the else statement to specify a block of code to be executed **if a condition is false**.

```
if (hour < 18) {  
  greeting = "Good day";  
} else {  
  greeting = "Good evening";  
}
```

- The JavaScript else if Statement

- Use the else if statement to specify a new condition if the first is false.

```
if (time < 10) {  
  greeting = "Good morning";  
} else if (time < 20) {  
  greeting = "Good day";  
} else {  
  greeting = "Good evening";  
}
```

- Switch Control Flow

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, no code is executed.

```
switch (new Date().getDay()) {  
  case 6:  
    text = "Today is Saturday";  
    break;  
  case 0:  
    text = "Today is Sunday";  
    break;  
  default:  
    text = "Looking forward to the Weekend";  
}
```

- The break / default Keyword

- break 키워드는 현재 실행 중인 case 블록을 종료하고 switch 문을 빠져나간다. 만약, break가 없으면 다음 case 블록이 연속적으로 실행된다.
- default는 주어진 값이 어떤 case와도 일치하지 않을 때 실행되는 코드 블록을 명세한다. default는 선택 사항이며 생략 가능하다.

- JavaScript loops repeat executing a block of code a number of times.

Loop Type	Description
for	Iterates over values and expressions
while	Iterates over a condition
do...while	Iterates over a condition
for...in	Iterates over the properties of an Object
for...of	Iterates over array like objects
forEach()	Iterates over each element in an Array

- for statement

```
for (exp 1; exp 2; exp 3) {  
  // code block to be executed  
}
```

- exp 1 is executed (one time) **before** the execution of the code block.
- exp 2 defines the **condition** for executing the code block.
- exp 3 is executed (every time) **after** the code block has been executed.

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

```
const arr = ["바나나", "사과", "자몽"];  
for(let i = 0; i < arr.length; i++){  
  console.log(arr[i]);  
}
```

- while statement

```
while (condition) {  
  // code block to be executed  
}
```

- 조건이 참(true)인 동안 반복

```
let count = 0;  
while (count < 3) {  
  console.log(count);  
  count++;  
}
```

- do while statement

```
do {  
  // code block to be executed  
}  
while (condition);
```

- 조건을 검사하기 전에 코드 블록을 한 번 실행한 후, 조건이 참(true)인 동안 반복

```
let num = 0;  
do {  
  console.log(num);  
  num++;  
} while (num < 3);
```

- break statement

- 특정 조건에서 반복을 중단해야 할 때 사용

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    break; // 반복문 종료  
  }  
  console.log(i);  
}
```

- continue statement

- 특정 조건에서 실행 코드를 건너뛰고 싶을 때 사용

```
for (let i = 0; i < 10; i++) {  
  if (i % 2 === 0) {  
    continue; // 짝수일 때는 건너뛰  
  }  
  console.log(i);  
}
```

- for...in statement

```
for (let key in 객체) {  
    // 실행 코드  
}
```

- 객체의 속성(key)을 반복할 때 사용

```
let obj = { name: '홍길동', age: 28, company: '활빈당' };  
for (let key in obj) {  
    console.log(key, obj[key]);  
}
```

- for...of statement

```
for (let value of 반복가능한객체) {  
    // 실행 코드  
}
```

- 반복 가능한 객체(배열, 문자열 등)의 엘리먼트 값을 반복

```
let arr = [10, 20, 30];  
for (let value of arr) {  
    console.log(value);  
}
```


- Nested Loop

```
for (let i = 0; i < 3; i++) {           // 외부 반복문
  for (let j = 0; j < 3; j++) {         // 내부 반복문
    console.log(`i: ${i}, j: ${j}`);
  }
}
```

- 반복 횟수 관리: 중첩 반복문은 실행 횟수가 기하급수적으로 늘어날 수 있으므로 조건과 횟수를 신중히 설정
- 가독성 유지: 중첩 깊이가 깊어지면 가독성이 떨어지므로, 가능한 깊이를 줄이거나 함수로 분리하여 관리

- What is Function and Why use Functions?
 - Functions are **reusable code blocks** designed to perform a particular task.
 - Functions are executed when they are called or invoked.
 - Functions help you to Reuse code (write once, run many times) and organize code into smaller parts
 - Functions help you to Make code easier to read and maintain

- JavaScript Function Syntax

```
function name( p1, p2, ... ) {  
  // code to be executed  
}
```

- A function can be created with the function keyword, a name, parentheses (), and brackets {}.
- To run a function, you call it by using its name followed by parentheses.
- A Function Can Be Used Many Times
- Variables declared within a JavaScript function, become LOCAL to the function.
- Functions can be used as variables, in all types of formulas, assignments, and calculations.

- Function Invocation

- The code inside a function will execute when "something" invokes the function
- When a function returns a value, you can store the value in a variable.
- You can call the same function whenever you need it.
- The () operator invokes a function.
- You can call functions from other functions, from events, or from any code block.

- Calling vs Referencing a Function

```
function sayHello() {  
  return "Hello World";  
}  
let text = sayHello;
```

- Referencing a Function: sayHello refers to the function itself. It returns the function.
- Calling a Function: sayHello() refers to the function result. It returns the result.

- Parameters allow you to pass (send) values to a function.
 - Parameters(매개변수) are the names listed in the function definition.
 - Arguments(인수) are the real values passed to, and received by the function.
- Parameter Rules
 - JavaScript function definitions do not specify data types for parameters.
 - JavaScript functions do not perform type checking on the arguments.
 - JavaScript functions do not check the number of arguments received.
→ Accessing a function with an incorrect parameter can return an incorrect answer
- Default Parameter Values
 - The default value is used if no argument is provided.

- The return Statement
 - When a function reaches a return statement, the function stops executing.
 - Most functions return the result of a calculation or an operation.
 - The returned value can be used anywhere a value is expected.
 - A function can return any type of value (not only numbers).
 - Code written after a return statement will NOT be executed.
 - If a function does not return a value, the return value will be undefined.
 - You can use return to stop a function early, based on a condition.

- Arguments are the real values passed to, and received by the function.
 - Arguments are assigned to parameters in the order they appear.
 - Arguments can be variables
 - If a function is called with fewer arguments than parameters, the missing values become undefined.
- Call by Value vs. Call by Sharing

구분	Call by Value (기본형 전달)	Call by Sharing (참조형 전달)
복사되는 대상	실제 데이터 값 (예: 10, "Hello")	메모리 주소 값 (예: 0x00412)
함수내부 수정	원본에 영향 없음	원본 객체의 프로퍼티 수정 시 영향 있음

```
function changeValue(x) {  
  // 복사본을 수정함  
  x = 20;  
}  
  
let a = 10;  
changeValue(a);  
  
console.log(a); // 10 (원본은 전혀 변하지 않음)
```

```
function changeProperty(obj) {  
  // 주소를 타고 들어가 내부 속성을 수정함  
  obj.name = "Lee";  
}  
  
let user = { name: "Kim" };  
changeProperty(user);  
  
console.log(user.name); // "Lee" (원본 객체가 변경됨!)
```

JS Functions - Function Expressions

- A function expression is a function stored in a variable

```
// Function Declaration (함수선언)
function multiply(a, b) {
  return a * b;
}

// Function Expression (함수표현식)
const multiply = function(a, b) {
  return a * b;
};
```

- Arrow Functions allow a shorter syntax for function expressions.

```
const multiply = (a, b) => a * b;
```

- Why use Function Expression?

- 함수 선언문은 '함수 Hoisting'이 발생하여 코드 최상단으로 끌어올려져, 선언하기도 전에 호출할 수 있다. 이는 편해 보이지만, 규모가 큰 프로젝트에서는 코드의 실행 흐름을 방해하고 엉뚱한 함수가 호출되는 버그를 유발한다.

```
// 1. 함수 선언문: 선언 전 호출 가능 (호이스팅 유발)
sayHello(); // "Hello"
function sayHello() { console.log("Hello"); }

// 2. 함수 표현식: 선언 전 호출 불가능 (안전함)
sayHi(); // ReferenceError: Cannot access 'sayHi' before initialization
const sayHi = function() { console.log("Hi"); };
```

- 함수 선언문은 자바스크립트 엔진이 코드를 실행하기 전 구문 분석 단계에서 미리 생성되므로, if문 같은 블록 안에서 유연하게 정의하기 까다롭지만 함수 표현식은 코드가 실행되는 시점(Runtime)에 평가되어 함수를 생성하므로, 상황에 따라 전혀 다른 함수를 변수에 할당할 수 있다.

```
let guestWelcome;
const isVIP = true;
// 조건에 따라 변수에 할당되는 '함수 값'을 동적으로 결정
if (isVIP) {
  guestWelcome = function() { console.log("VIP 전용 라운지로 안내합니다."); };
} else {
  guestWelcome = function() { console.log("일반 대기실로 안내합니다."); };
}
guestWelcome(); // 실행 시점에 결정된 함수가 유연하게 호출됨
```

- 특히, 화살표 함수는 극도로 간결해지는 장점이 있다.

- An Array is an object type designed for storing data collections.
- Key characteristics of JavaScript arrays are:
 - **Elements:** An array is a list of values, known as elements.
 - **Ordered:** Array elements are ordered based on their index.
 - **Zero indexed:** The first element is at index 0, the second at index 1, and so on.
 - **Dynamic size:** Arrays can grow or shrink as elements are added or removed.
 - **Heterogeneous:** Arrays can store elements of different data types (numbers, strings, objects and other arrays).

```
const cars = ["Saab", "Volvo", "BMW"];
```

JS Arrays - Creating and Accessing an Array

JavaScript

- Creating an Array

- Using an array literal

```
const cars = ["Saab", "Volvo", "BMW"];
```

- Using the JavaScript Keyword new

```
const cars = new Array("Saab", "Volvo", "BMW");
```

- Accessing and Changing an Array Element

- You access an array element by referring to the index number

```
let car = cars[0];
```

- This statement changes the value of the first element in cars:

```
cars[0] = "Opel";
```

- The length Property

- The length property of an array returns the length of an array (the number of array elements).

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
let length = fruits.length;
```

- Basic Array Methods
 - **toString()**, **at()**, **join()**, **pop()**, **push()**, **shift()**, **unshift()**, **isArray()**, **delete()**, **concat()**, **copyWithin()**, **flat()**, **slice()**, **splice()**, **toSpliced()**
 - **toString()** -returns the elements of an array as a comma separated string.
 - **pop()** - removes the last element from an array
 - **push()** - adds a new element to an array (at the end)
- Array Search Methods
 - **indexOf()**, **lastIndexOf()**, **includes()**, **find()**, **findIndex()**, **findLast()**, **findLastIndex()**
 - **indexOf()** - searches an array for an element value and returns its position.
 - **lastIndexOf()** - is the same as **Array.indexOf()**, but returns the position of the last occurrence of the specified element.
 - **includes()** - allows us to check if an element is present in an array.

- Array Sorting Methods
 - **sort(), reverse(), toSorted(), toReversed()**
 - sort() - sorts an array alphabetically:
 - reverse() - reverses the elements in an array:
 - toSorted() - sort an array without altering the original array.
 - toReversed() - reverse an array without altering the original array.
- Array Iteration Methods
 - **forEach(), map(), flatMap(), filter(), reduce(),** reduceRight(), every(), some(), from(), keys(), entries(), with()
 - forEach() - calls a function (a callback function) once for each array element.
 - filter() - creates a new array with array elements that pass a test.
 - reduce() - runs a function on each array element to produce a single value.
- Spread Operator(...)
 - The ... operator expands an array into individual elements

```
const numbers = [23, 55, 21, 87, 56];  
let minValue = Math.min(...numbers);
```

 - Math.min(...numbers) → Math.min(23, 55, 21, 87, 56)으로 변환되어 처리됨.

- What are JavaScript Objects?

- Objects are containers for Properties and Methods
- Properties are named Values stored as key:value pairs
- Methods are Functions stored as key:function() pairs.

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  age      : 50,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

- How to Create a JavaScript Object

- An **object literal** is the simplest and most common way to define a JavaScript object.

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

- Create a new JavaScript object using **new Object()**:

```
const person = new Object();  
person.firstName = "John";  
person.lastName = "Doe";  
person.age = 50;  
person.eyeColor = "blue";
```

- You can access object properties in these ways
 - Dot notation (preferred)
 - Bracket notation (Bracket notation is necessary in some cases)
 - Expression
- Add, change, and delete properties using assignment and delete
 - You can change the value of a property
 - You can add a new property by simply giving it a value
 - The delete keyword deletes a property from an object
 - Use the in operator to check if a property exists in an object
- Nested Objects
 - Property values in an object can be other objects

- The this Keyword
 - In an object method, this refers to the object.
- Accessing Object Methods
 - To call an object method, add parentheses ()
- Adding a Method to an Object
 - You can add a method to an object by assigning a function to a property

- Displaying a JavaScript object will output [object Object].
- How to Display JavaScript Objects?
 - You can add a method to an object by assigning a function to a property
 - The properties of an object can be collected in a For .. In loop
 - Object.values() creates an array from the property values.
 - Object.entries() makes it simple to use objects in loops.
 - JavaScript objects can be converted to a string with JSON method JSON.stringify().

- Object Constructor Function

- To create an object type we use an object constructor function.

```
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eye;  
}  
  
const myFather = new Person("John", "Doe", 50, "blue");  
const myMother = new Person("Sally", "Rally", 48, "green");  
const mySister = new Person("Anna", "Rally", 18, "green");
```

- Constructor Function **prototype**

- To add a new property, you must add it to the constructor function **prototype**
- Adding a new method to a constructor must be done to the constructor function **prototype**

- Hoisting is JavaScript's default behavior of moving declarations to the top.
 - JavaScript Declarations are Hoisted
 - Variables defined with `let` and `const` are hoisted to the top of the block, but not initialized.
 - JavaScript Initializations are Not Hoisted
- Declare Your Variables At the Top !
 - Hoisting is (to many developers) an unknown or overlooked behavior of JavaScript.
 - If a developer doesn't understand hoisting, programs may contain bugs (errors).
 - To avoid bugs, always declare all variables at the beginning of every scope.
 - Since this is how JavaScript interprets the code, it is always a good rule.

- JavaScript Date Objects let us work with dates

```
const d = new Date();  
const d = new Date("2022-03-25");
```

- `new Date()` creates a date object with the current date and time.
- `new Date(date string)` creates a date object from a date string.
- JavaScript Stores Dates as Milliseconds
 - JavaScript stores dates as number of milliseconds since January 01, 1970.
- Displaying Dates
 - `toString()`
 - `toDateString()`
 - `toUTCString()`
 - `toISOString()`

- JavaScript Set
 - A JavaScript Set is a collection of unique values.
 - Each value can only occur once in a Set.
- How to Create a Set
 - Passing an array to new Set()
 - Create an empty set and use add() to add values

- JavaScript Map
 - A JavaScript Map is an object that can store collections of **key-value** pairs
- How to Create a Map
 - Create a new Map and add elements with Map.set()
 - Passing an existing Array to the new Map() constructor

- JavaScript Math
 - The JavaScript Math object allows you to perform mathematical tasks.
 - The Math object is static.
 - All methods and properties can be used without creating a Math object first.
- The syntax for any Math property is : Math.property.
- The syntax for Math any methods is : Math.method(number)
 - https://www.w3schools.com/js/js_math_reference.asp

- Regular Expression (Regex)
 - A Regular Expression(Regex) is a sequence of characters that forms a search pattern.
 - JavaScript RegExp is an Object for handling Regular Expressions.
 - RegExp are be used for Text searching, Text replacing and Text validation

- Regular Expression Syntax

```
/pattern/modifier flags
```

- Example: /w3schools/i is a regular expression

```
let n = text.search(/w3schools/i);
```
- w3schools is a pattern (to be used in a search).
- i is a modifier (modifies the search to be case-insensitive).

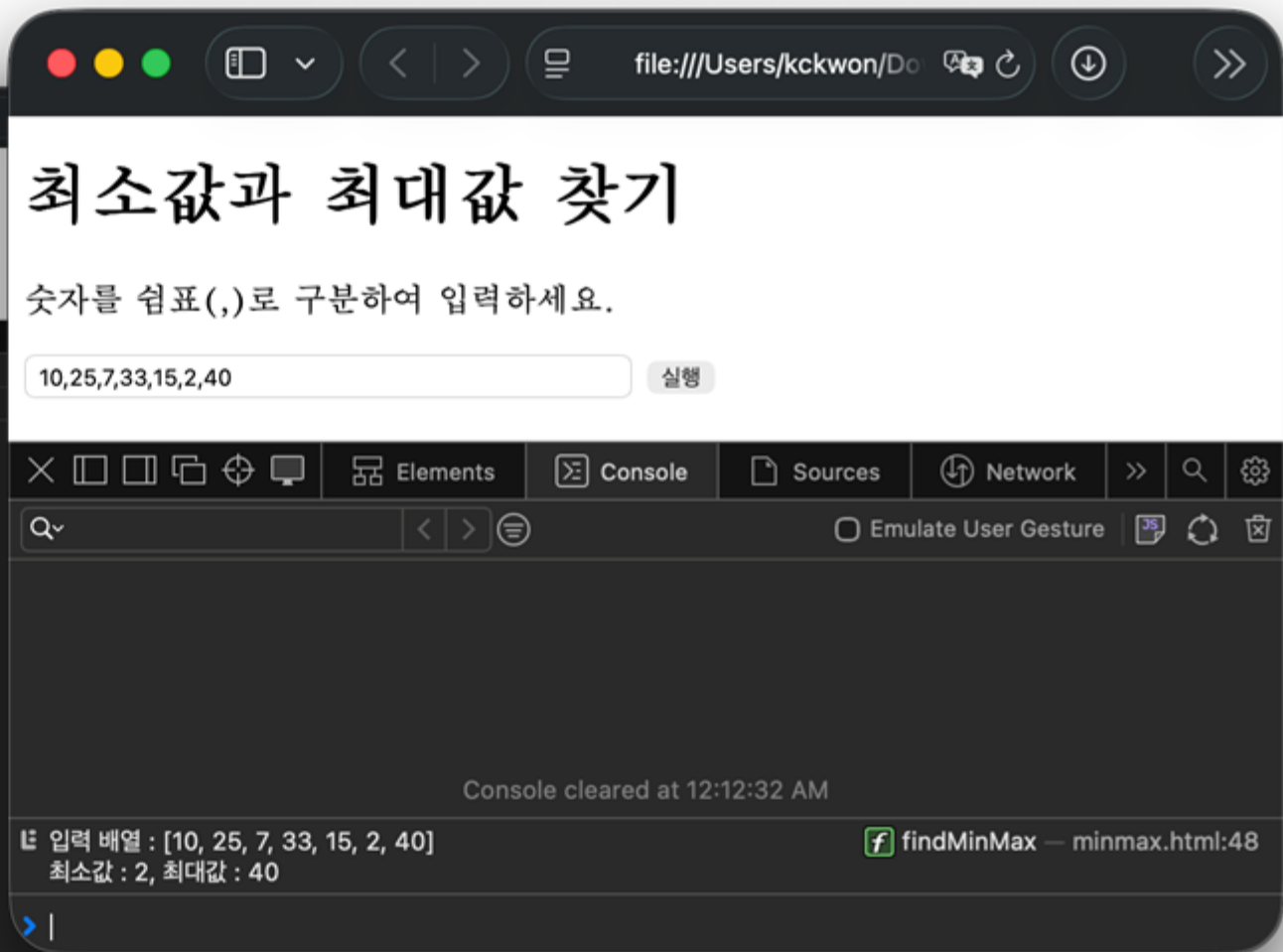
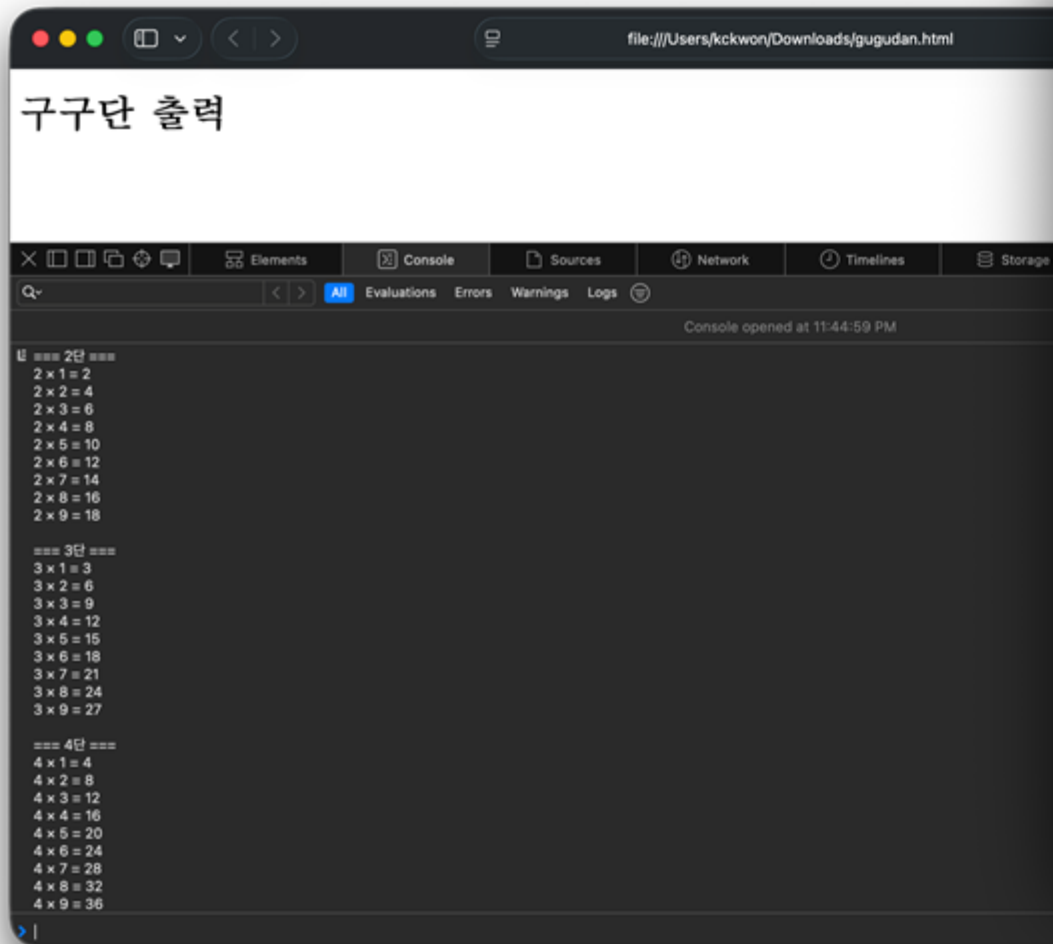
- Phone Number Validation

```
const regex = /[0-9]{3}-[0-9]{4}-[0-9]{4}/;  
const phone = "010-1234-5678";  
console.log(regex.test(phone)); // true
```

[실습] JavaScript

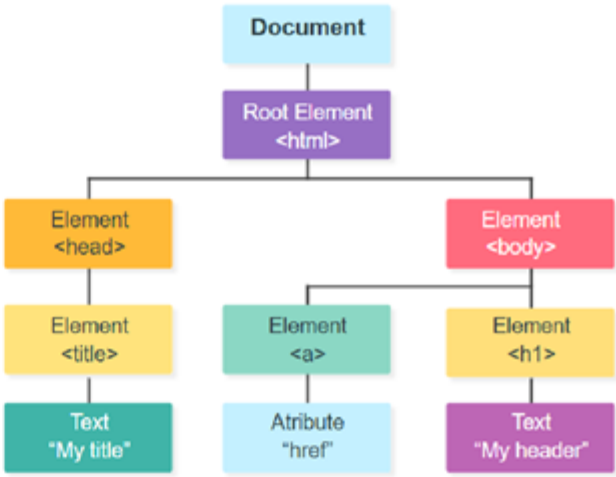
JavaScript

연산자, 반복문, 함수 등을 사용



- HTML Document Object Model
 - 웹 문서(HTML, XML)를 구조화하여 브라우저나 프로그래밍 언어에서 쉽게 접근하고 조작할 수 있도록 하는 표준화된 인터페이스
 - DOM은 웹 문서를 트리 구조로 표현하며, 문서의 각 요소를 객체로 간주
 - W3C(World Wide Web Consortium)와 WHATWG(Web Hypertext Application Technology Working Group)에 의해 정의된 표준
 - 자바스크립트를 통해 DOM 요소를 실시간으로 변경 가능
 - 클릭, 입력, 스크롤과 같은 사용자 이벤트 처리 가능
- DOM Tree
 - When a web page loads, the browser creates a tree-like representation of the HTML document.

Node	Description
Document	Owner of all nodes in the document
<html>	Element Node
<head>	Element Node
<body>	Element Node
<a>	Element Node
href	Attribute Node
<h1>	Element Node
My Header	Text Node



- HTML DOM API (Application Programming Interface)
 - The DOM API is a set of Methods and Properties that allow JavaScript to change the content, structure, and style of any HTML elements.
- Example

```
<html>
  <body>
    <p id="demo"></p>

    <script>
      const myPara = document.getElementById("demo");
      myPara.innerHTML = "Hello World!";
    </script>

  </body>
</html>
```

- document is the HTML document.
- getElementById() is a document method.
- myPara = getElementById("demo") retrieves the "demo" element.
- innerHTML is an element property.
- myPara.innerHTML = "Hello World!" changes the property.

HTML DOM - Selecting Elements

JavaScript

- Finding HTML Elements
 - Finding HTML elements by id
 - Finding HTML elements by tag name
 - Finding HTML elements by class name
 - Finding HTML elements by CSS selectors
 - Finding HTML elements by HTML object collections
- Finding HTML Elements by HTML Object Collections

- The HTML DOM allows JavaScript to change both the text and the content of HTML elements.
 - *innerHTML* : getting or replacing the content of HTML elements
 - *attribute* : change the value of an HTML attribute
 - *style.property* : change the style of an HTML element
- document.write()
 - document.write() can be used to write directly to the HTML output stream:

- HTML Events

- HTML events are things that happen to HTML elements.
- Ex) An HTML button is clicked, A web page has finished loading, The mouse moves over an element, A keyboard key is pressed, An HTML input field is changed

- JavaScript Events

- When JavaScript is used in HTML pages, JavaScript can react on events.
- JavaScript lets you execute code when events are detected.

```
<element event="some JavaScript">
```

- Ex)

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The time is?</button>  
<button onclick="this.innerHTML = Date()">The time is?</button>
```

- It is more common to use the event attribute to call a function:

```
<button onclick="displayDate()">The time is?</button>  
  
<script>  
function displayDate() {  
    document.getElementById("demo").innerHTML = Date();  
}  
</script>
```

- 주요 이벤트 목록

이벤트	설명	예시
click	사용자가 요소를 클릭할 때 발생	<code><button onclick="alert('Button clicked!')">Click Me</button></code>
dblclick	사용자가 요소를 두 번 클릭할 때 발생	<code><div ondblclick="alert('Double clicked!')">Double Click Me</div></code>
mouseover	마우스 포인터가 요소 위에 올려졌을 때 발생	<code><div onmouseover="alert('Mouse over!')">Hover me</div></code>
mouseout	마우스 포인터가 요소에서 벗어났을 때 발생	<code><div onmouseout="alert('Mouse out!')">Mouse Out Me</div></code>
keydown	키보드의 키가 눌릴 때 발생	<code><input type="text" onkeydown="alert('Key down!')"></code>
keyup	키보드에서 키가 떼어질 때 발생	<code><input type="text" onkeyup="alert('Key up!')"></code>
change	입력 값이나 선택 값이 변경될 때 발생	<code><input type="text" onchange="alert('Value changed!')"></code>
focus	요소가 포커스를 받았을 때 발생	<code><input type="text" onfocus="alert('Focused!')"></code>
blur	요소의 포커스가 벗어났을 때 발생	<code><input type="text" onblur="alert('Blurred!')"></code>
load	페이지나 이미지가 완전히 로드된 후 발생	<code></code>
unload	페이지가 언로드될 때 발생	<code><body onunload="alert('Page unloaded!')"></code>

- An event handler is JavaScript code that runs when an event happens

방식	설명	비고
HTML 속성 방식 (Not Recommended)	HTML 태그 내부에 직접 이벤트 속성(onclick, onmouseover 등)을 사용하여 JS 함수를 연결하는 방식	- JS와 HTML의 역할 분리 원칙 위배 - 유지보수 어려움 - 복잡한 로직 작성에 부적합
DOM 프로퍼티 방식 (Not Recommended)	JavaScript에서 DOM 요소의 이벤트 속성(예: onclick)에 함수를 할당하여 이벤트를 처리 (전통적인 방식)	- 이벤트 핸들러 1개만 등록 가능 - 다른 코드에서 덮어쓸 수 있어 충돌 위험
Event Listener 활용 (Highly Recommended)	addEventListener() 메서드를 사용하여 이벤트 핸들러를 등록하는 방식 - 권장되는 방식	- 복수 핸들러 등록 가능 - 버블링/캡처 제어 가능 - 유지보수 및 확장성 우수

[쉽지만 비권장]

```
<button onclick="displayDate()">Time is?</button>

<p id="demo"></p>

<script>
// Function to display date
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>
```

[어렵지만 권장]

```
<button id="myBtn">Click me</button>

<p id="demo"></p>

<script>
const btn = document.getElementById("myBtn");

// Add EventListener to btn
btn.addEventListener("click", function () {
  document.getElementById("demo").innerHTML = Date();
});
</script>
```

- Mouse Events : https://www.w3schools.com/js/js_events_mouse.asp
- Keyboard Events : https://www.w3schools.com/js/js_events_keyboard.asp
- Load Events : https://www.w3schools.com/js/js_events_load.asp
- Timing Events : https://www.w3schools.com/js/js_events_timing.asp
- 기타Events : https://www.w3schools.com/js/js_html_dom_events.asp

종합 실습 진행